

Heap / Priority Queue — Fundamentals, Top K and the 8 Patterns

What this is. A complete study guide built from a guided, question-by-question session on heaps. It keeps **everything** from that session — the fundamentals, the *thought process* behind every pattern, every problem and its solution, the recognition rules, the Python templates and the harder follow-up problems — and adds diagrams, traced dry-runs, complexity notes and interview questions.

The one idea to keep. Don't memorise "*K largest* → *min-heap*". Ask instead: "**What information do I repeatedly need from a changing set — and can a heap make exactly that operation cheap?**"

How the guide is organised

Part	Content
0	Roadmap and pattern map
1	Heap fundamentals — what a heap is, operations, <code>heapq</code> in Python
2	Pattern 1 — Top K (K best candidates)
3	Pattern 2 — Two Heaps (running median)
4	Pattern 3 — Repeatedly process the min / max
5	Pattern 4 — K-way merge
6	Pattern 5 — Sort + Heap (intervals, scheduling)
7	Pattern 6 — Heap + Greedy
8	Pattern 7 — Lazy deletion (stale entries)
9	Pattern 8 — Hash map + Heap (composite)
10	Templates to memorise, decision guide, combinations, pitfalls, cheat sheets, practice list, interview Q&A

Every pattern section follows the same shape: **recognition signal** → **derive it (the thought process)** → **dry run** → **code** → **complexity** → **mental model** → **harder problems**. The code was executed and checked against brute-force solutions on thousands of random inputs; the step tables are produced by really running the algorithms.

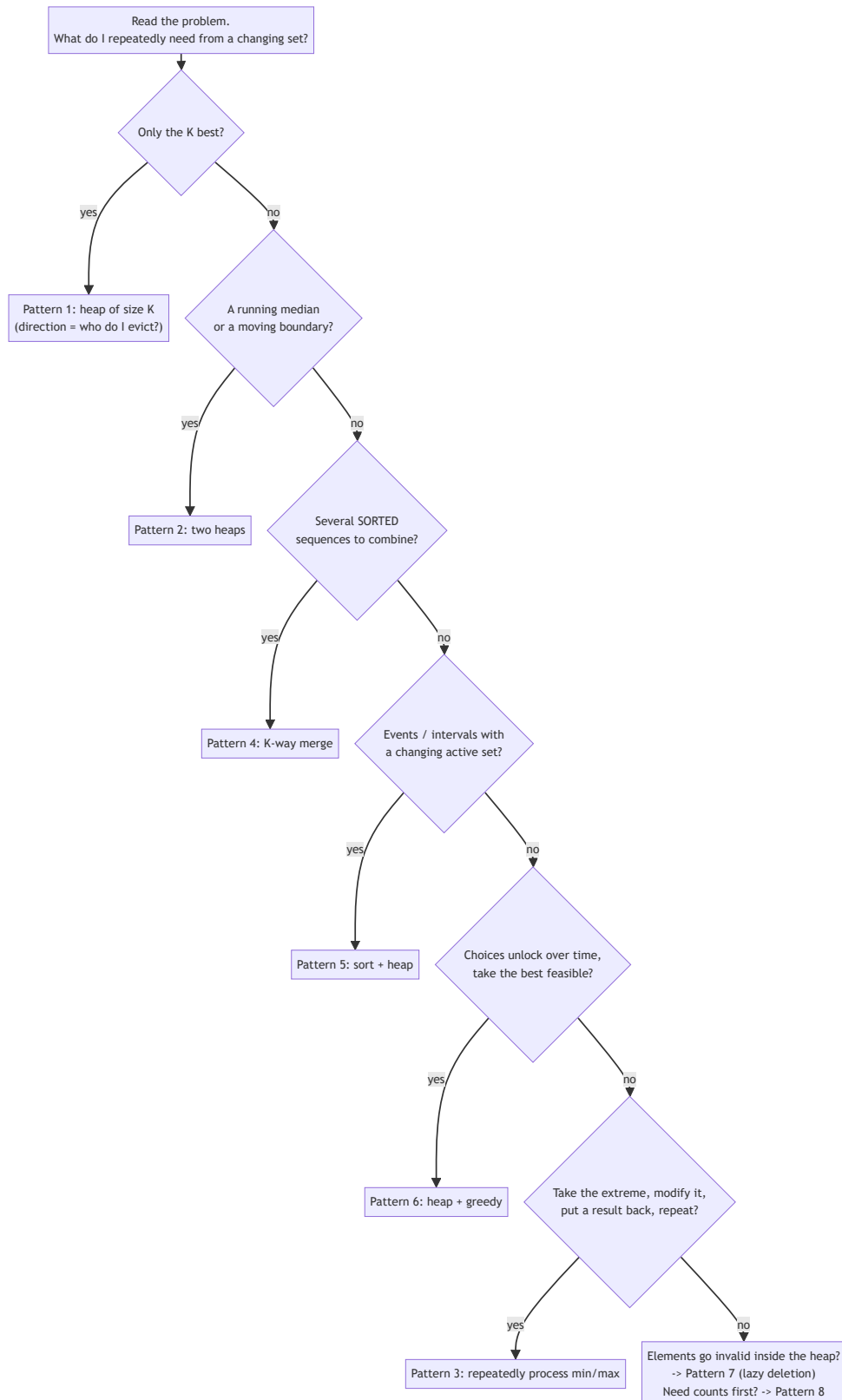
Corrections to the original session. While verifying every program, three of the harder solutions in the source turned out to be wrong: *Sliding Window Median* (wrong or crashing on about a third of random tests), *Furthest Building You Can Reach* (spent the ladder on the wrong climb) and *Task Scheduler* (a sign mix-up that makes it return the wrong time). They are corrected in this guide, each marked **Correction**, with the failing case shown.

Part 0 · Roadmap

0.1 The eight patterns at a glance

#	Pattern	Recognition signal	Heap	Problems in this guide
1	Top K	"I only need the best K , not the whole order"	size-K heap; the root is the <i>weakest</i> of the K	Kth largest, Kth smallest, Top K frequent, K closest points, K smallest pairs, Kth in sorted matrix
2	Two heaps	"median / a boundary that keeps moving"	max-heap (lower half) + min-heap (upper half)	Median of a stream, Sliding window median
3	Repeatedly process min/max	"take the current best, change the set, repeat"	one heap holding <i>all remaining</i> items	Connect ropes, Last stone weight, Min cost to hire K workers, Furthest building
4	K-way merge	"K sorted sources, need the next smallest across them"	min-heap with one candidate per source	Merge K arrays / lists, Smallest range, K pairs, sorted matrix
5	Sort + Heap	"events arrive in order; a changing set of <i>active</i> items"	sort by arrival, min-heap of <i>end</i> times	Meeting rooms II, Minimum servers, CPU scheduling
6	Heap + Greedy	"choices become feasible over time; take the best feasible one"	max-heap of feasible candidates	IPO, Max performance of a team, Min refuelling stops
7	Lazy deletion	"items become invalid while inside the heap"	heap + validity check at the top	Sliding window maximum, Dijkstra
8	Hash map + Heap	"count first, then pick the best / block the last one"	max/min-heap over counts (+ a cooldown queue)	Top K frequent, Reorganize string, Task scheduler

0.2 First-glance decision tree



These patterns combine. Real problems rarely announce their pattern: *Sliding Window Median* is two heaps **plus** lazy deletion; *K smallest pairs* is K-way merge **plus** Top K; *IPO* is sort **plus** heap **plus** greedy. Part 10 lists the combinations.

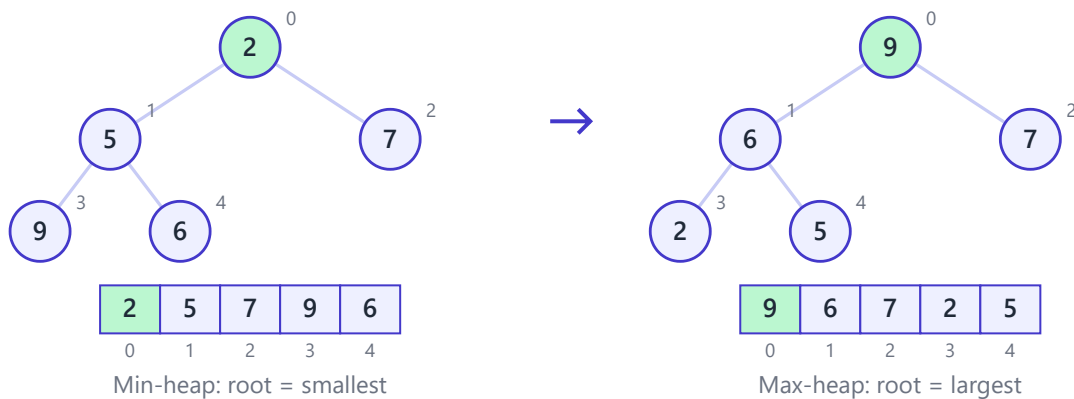
Part 1 · Heap Fundamentals — the minimum you need

1.1 What a heap is

Think of a heap as **a structure that gives you fast access to one extreme of a set**.

- **Min-heap** — the smallest element is always at the root.
- **Max-heap** — the largest element is always at the root.

A heap only guarantees the **ROOT** is the extreme - the rest is not sorted



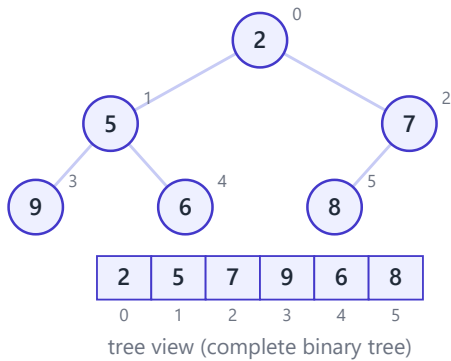
The important thing is not that the whole structure is sorted. It is only guaranteed that **the root is the minimum (or maximum)**. In a min-heap every parent is $\leq$ its children; nothing is promised about siblings or cousins (in the min-heap above, **6 sits deeper than 7 yet is smaller than 7** — that is perfectly legal).

The precise rule is the **heap property**: for a min-heap, $\text{parent} \leq \text{child}$ for every parent/child pair (for a max-heap, $\text{parent} \geq \text{child}$). And a heap is always a **complete binary tree**: every level is full except possibly the last, which is filled from the left.

1.2 Tree view and array view

Because the tree is *complete*, it needs no pointers. It is stored in a plain array, level by level, and parent/child positions come from arithmetic:

Same heap, two views: tree and array



Index arithmetic (0-based)

parent(i) $(i - 1) // 2$

left(i) $2i + 1$

right(i) $2i + 2$

Example: node 4 (value 6) -> parent = $(4-1)//2 = 1$ (value 5)

Example: node 1 (value 5) -> children = 3 and 4 (values 9, 6)

1-based arrays use $i//2$, $2i$, $2i+1$ instead.

For node at index i	0-based	1-based
parent	$(i - 1) // 2$	$i // 2$
left child	$2i + 1$	$2i$
right child	$2i + 2$	$2i + 1$

This is why a heap is compact and cache-friendly, and why `heap[0]` **is the root** in Python.

Height. A complete binary tree with n nodes has height $\text{floor}(\log_2 n)$. Every heap operation walks at most one root-to-leaf path, which is why they cost **$O(\log n)$** .

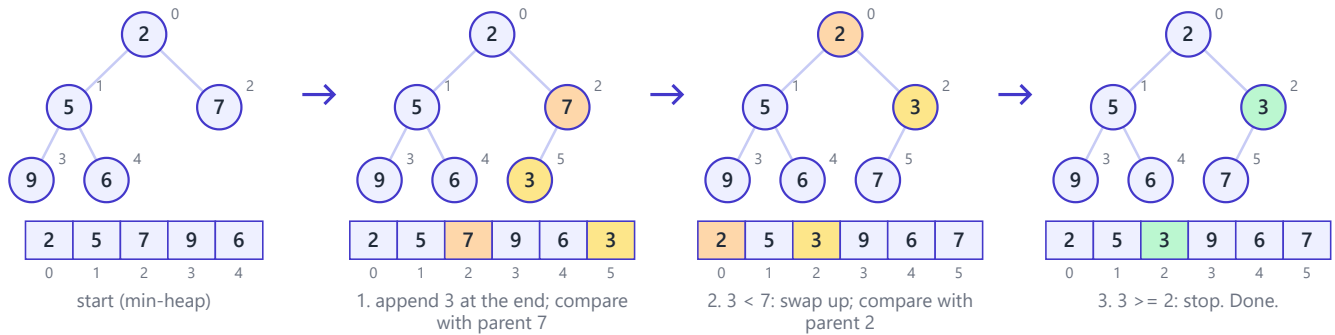
1.3 Operations and complexity

Operation	Complexity	Why
Peek (look at min/max)	$O(1)$	the root is directly accessible (<code>heap[0]</code>)
Insert (push)	$O(\log n)$	append at the end, then <i>sift up</i> — moves up at most the height
Extract min/max (pop)	$O(\log n)$	replace the root with the last item, then <i>sift down</i> — moves down at most the height
Heapify (build from an array)	$O(n)$	bottom-up construction; most nodes are near the leaves and barely move
Build by n inserts	$O(n \log n)$	the slower way — use <code>heapify</code> instead
Search for an arbitrary value / delete an arbitrary value	$O(n)$	a heap is not built for this (this is what Pattern 7 works around)

Insert — sift up

Append the new value at the end (the next free spot of the complete tree) and let it *bubble up* while it is smaller than its parent.

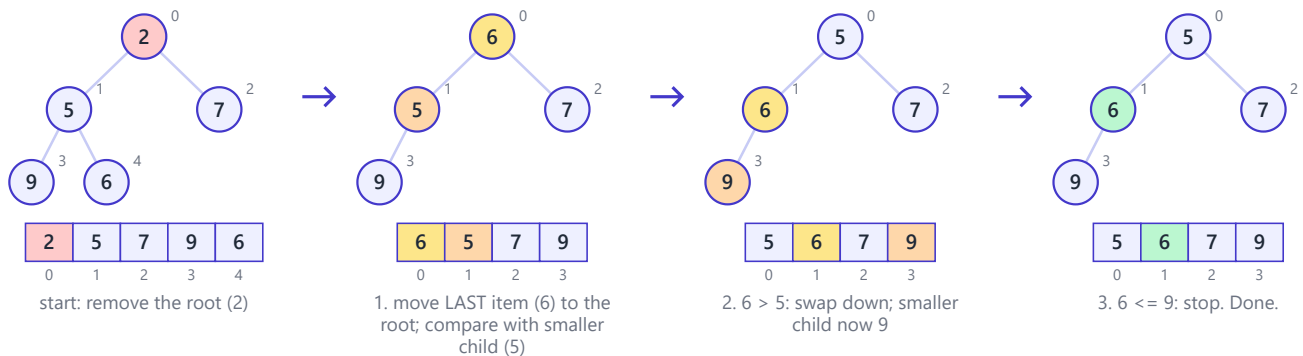
INSERT 3 into a min-heap: append, then sift UP (at most height = $\log n$ swaps)



Extract-min — sift down

Take the root out, move the **last** item into the root, and let it *sink down*, each time swapping with the **smaller** child (for a min-heap) until the heap property holds again.

EXTRACT-MIN: replace root with the last item, then sift DOWN (swap with the SMALLER child)

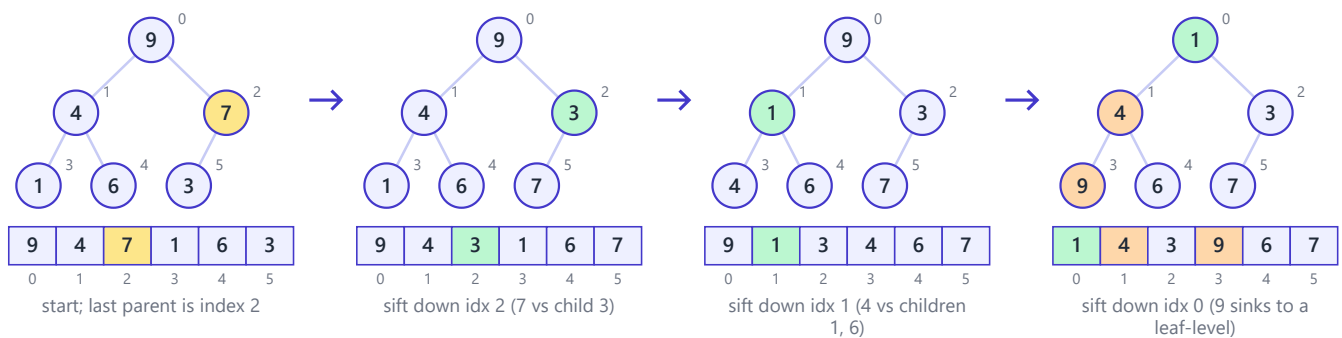


Why the *smaller* child? If we swapped with the larger child, that larger child would become the parent of the smaller one and break the heap property.

Heapify — why $O(n)$ and not $O(n \log n)$

Start at the **last parent** (index $n/2 - 1$) and sift each parent down, moving toward the root. Leaves (about half the nodes) need no work.

HEAPIFY (bottom-up, min-heap): sift down every parent from the last one to the root $\rightarrow O(n)$



The cost adds up like this: about $n/2$ nodes are leaves (0 swaps), $n/4$ nodes can sink at most 1 level, $n/8$ nodes at most 2, and so on:

$$n/4 \cdot 1 + n/8 \cdot 2 + n/16 \cdot 3 + \dots = n \cdot \sum k / 2^{(k+1)} = O(n)$$

Only the *few* nodes near the top can sink far; the *many* nodes near the bottom sink only a little. That is why building a heap is linear, while n separate inserts would be $O(n \log n)$.

1.4 Heap vs the alternatives

Structure	Peek min	Insert	Delete min	Find arbitrary	Build
Unsorted array	$O(n)$	$O(1)$	$O(n)$	$O(n)$	–
Sorted array	$O(1)$	$O(n)$	$O(1)$ if the min is kept at the end, else $O(n)$	$O(\log n)$	$O(n \log n)$
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n \log n)$
Heap	$O(1)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$

A heap is the right tool when **the set keeps changing while you repeatedly need its minimum/maximum**. If you only need the sorted order once, plain sorting is fine — Pattern 3 comes back to this.

1.5 Python's `heapq` (a min-heap)

```
import heapq

heap = []
heapq.heappush(heap, x)           # insert                 $O(\log n)$ 
x = heapq.heappop(heap)          # remove and return min  $O(\log n)$ 
x = heap[0]                      # peek the minimum        $O(1)$ 
heapq.heapify(arr)               # turn a list into a heap, in place  $O(n)$ 
```

A max-heap — `heapq` has no max-heap, so store **negative** values and negate again when reading:

```
heapq.heappush(heap, -x)         # insert  $x$  into a max-heap
x = -heapq.heappop(heap)         # remove the largest
largest = -heap[0]               # peek the largest
```

(Python 3.14 adds dedicated `*_max` functions, `heapq.heappush_max` and friends; on older versions use the negation trick.)

Other useful `heapq` functions

Function	What it does
<code>heapq.heappushpop(heap, x)</code>	push x then pop the smallest — faster than doing them separately; if x is smaller than the root it is returned straight away
<code>heapq.heapreplace(heap, x)</code>	pop the smallest then push x (the heap must be non-empty); the popped value may be larger than x
<code>heapq.nlargest(k, iterable, key=...)</code> / <code>nsmallest(k, ...)</code>	the K largest / smallest, $O(n \log k)$; convenient for one-off use
<code>heapq.merge(*sorted_iterables)</code>	lazily merges already-sorted inputs — K -way merge (Pattern 4) built in

Tuples as entries. A heap of tuples compares element by element, so *(priority, payload)* works naturally:

```
heapq.heappush(heap, (distance, node))    # ordered by distance
heapq.heappush(heap, (-dist, x, y))      # max-heap by distance
```

Two things to remember:

1. **Ties fall through to the next tuple element.** If that element is not comparable (for example a `ListNode` object), Python raises `TypeError` — add a unique tie-breaker such as an index: `(node.val, i, node)`.
2. **Negate the sort key, not the whole tuple**, when you want a max-heap on the first field.

Invariant — the word that keeps coming back. An *invariant* is a condition you keep true throughout the algorithm — the rule that says **what the heap currently represents**. For "K largest" the invariant is "*the min-heap contains the K largest elements seen so far*"; for Last Stone Weight it is "*the max-heap contains exactly the stones still remaining*". Instead of asking "*why am I doing these heap operations?*", ask "*what must my heap represent right now?*" Every operation should preserve or restore that meaning.

Part 2 · Pattern 1 — Top K (K best candidates)

Recognition. "I only care about the best K elements, not the whole sorted order." **The key question.** Among the K candidates I'm keeping, which one is the weakest and should be kicked out?

Don't memorise "Top K $\rightarrow$ heap". Derive it.

2.1 Deriving it: Kth Largest Element (LeetCode 215)

Problem. Given an unsorted array `nums` and an integer `k`, return the k -th largest element. $1 \leq k \leq \text{nums.length}$; `nums.length` can be large; you do **not** need the sorted array. Example: `nums = [3, 2, 1, 5, 6, 4]`, `k = 2` $\rightarrow$ 5.

Step 1 — Forget heaps. What is the obvious approach, and what information do we actually need? Sort, then read off the answer:

```
[3, 2, 1, 5, 6, 4] --sort descending--> [6, 5, 4, 3, 2, 1]
                                         ^ 2nd largest
```

Sorting costs $O(n \log n)$. But we do not care about the whole sorted array — **only the 2 largest elements**.

Step 2 — Let the numbers arrive one by one ($3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 6 \rightarrow 4$) and keep only **2 candidates**.

- After `3, 2`: candidates = `[3, 2]`.
- `1` arrives. Do we need to keep it? **No — it is smaller than both**. Anything smaller than both current candidates can never become part of the final top 2.
- `5` arrives, larger than both. Now we hold `[3, 2, 5]` and must drop one to keep only the best 2. **Which one is easiest to remove?** The **weakest of the current top 2 — the smallest, 2**. We keep `[5, 3]`.

Step 3 — Where does the heap come in? We keep having to answer: "which of my current K candidates is the weakest?" For **K largest** the weakest candidate is the **smallest** one. So we need cheap access to the **minimum** of the K candidates $\rightarrow$ a **min-heap**.

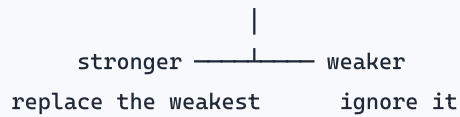
Which heap for Top-K? Ask: "which of my K candidates is the weakest?"



The mental model. For K largest, maintain a min-heap of size K . The root is the weakest member of the current top K .

Notice we did **not** choose a min-heap because "K largest uses min-heap". We chose it because *we repeatedly need the smallest among the K candidates*.

new candidate → compare with heap root



K largest with a min-heap, k = 2: nums = [3, 2, 1, 5, 6, 4]



green = enters the top-K and pushes out the weakest; red = weaker than the root, ignored; answer = root = 5

Dry run (k = 2 , min-heap, root first):

arrives	decision	min-heap (root first)	root
3	heap not full: push	[3]	3
2	heap not full: push	[2, 3]	2
1	1 <= root 2: ignore	[2, 3]	2
5	5 > root 2: evict 2, insert 5	[3, 5]	3
6	6 > root 3: evict 3, insert 6	[5, 6]	5
4	4 <= root 5: ignore	[5, 6]	5

The answer is the heap root: 5.

2.2 The mirror image: K smallest

Suppose the problem asks for the **K smallest** elements. What should the heap contain — min-heap or max-heap?

Goal: keep the K smallest

↓

Which current candidate is the weakest?

↓

the LARGEST one

↓

need fast access to the largest

↓

MAX-HEAP

Goal	Heap	Root represents
K largest	min-heap	weakest of the K largest
K smallest	max-heap	weakest of the K smallest

Worked reasoning. $K = 3$ and we currently hold the 3 smallest $[2, 4, 7]$. A new 5 arrives $\rightarrow [2, 4, 7, 5]$. Now 7 is the weakest (largest of the four) — a max-heap hands us 7 immediately, we remove it and 5 takes its place.

2.3 Insert first, or compare first?

Question. $K = 3$ largest, heap = $[2, 5, 8]$, new element 6 arrives. Should we (A) insert 6 and then remove the root, (B) ignore 6 , or (C) replace 8 directly? **Answer: A.** 6 is larger than the weakest candidate (2), so it could belong in the top 3 . Insert it $\rightarrow [2, 5, 8, 6]$, we now have 4 elements but want $3 \rightarrow$ remove the root (2) $\rightarrow [5, 6, 8]$.

The insight. For K largest, if a new element is larger than the weakest current candidate, it enters the heap and pushes out the weakest.

A small optimisation — check the root first, so an element that clearly can't qualify is never inserted:

```
if x > heap[0]:
    heapq.heappop(heap)
    heapq.heappush(heap, x)      # or, in one step: heapq.heapreplace(heap, x)
```

2.4 Complexity

With n numbers and a heap of size k , for every number we do at most:

Step	Cost	Why
peek	$O(1)$	root access
pop	$O(\log k)$	the heap holds only K elements, not N , so its height is $\log k$
push	$O(\log k)$	same

n elements $\rightarrow O(n)$ iterations $\rightarrow$ each heap operation $O(\log k) \rightarrow O(n \log k)$

- **Time: $O(n \log k)$ · Space: $O(k)$**
- Compare with sorting: **$O(n \log n)$** . When k is much smaller than n , and especially when the data is a *stream* or too large to hold in memory, the heap wins.

Approach	Time	Space	Notes
Sort everything	$O(n \log n)$	$O(1) - O(n)$	simple; needs all data at once
Heap of size K	$O(n \log k)$	$O(k)$	works on streams; best when $k \ll n$
Quickselect	$O(n)$ average, $O(n^2)$ worst	$O(1)$	fastest on average for a <i>static</i> array; not for streams

2.5 Recognition — think, don't recite

Don't memorise " K largest $\rightarrow$ min-heap." Instead think:

1. "I only need K elements, so can I maintain those K candidates instead of sorting everything?"
2. "Among my K candidates, which one might need to be kicked out?" — **that** determines the heap direction.

2.6 Test yourself: the 3rd smallest element

`nums = [7, 10, 4, 3, 20, 15]` , `k = 3` . What would you maintain and which heap?

Reasoning. Maintain the **3 smallest** elements; the weakest of those (the largest) must be at the root → **max-heap**.
Invariant: *the max-heap holds the 3 smallest elements seen so far.*

arrives	decision	max-heap (root first)	root
7	heap not full: push	[7]	7
10	heap not full: push	[10, 7]	10
4	heap not full: push	[10, 7, 4]	10
3	3 < root 10: evict 10, insert 3	[7, 3, 4]	7
20	20 >= root 7: ignore	[7, 3, 4]	7
15	15 >= root 7: ignore	[7, 3, 4]	7

Final heap root = **7**, so the 3rd smallest is **7**. (**20** and **15** are both larger than the current weakest **7**, so they can never be among the 3 smallest.)

```
import heapq

def find_kth_largest(nums, k):
    heap = []                                # min-heap holding the K largest seen so far
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)              # evict the weakest (smallest)
    return heap[0]                           # root = K-th largest

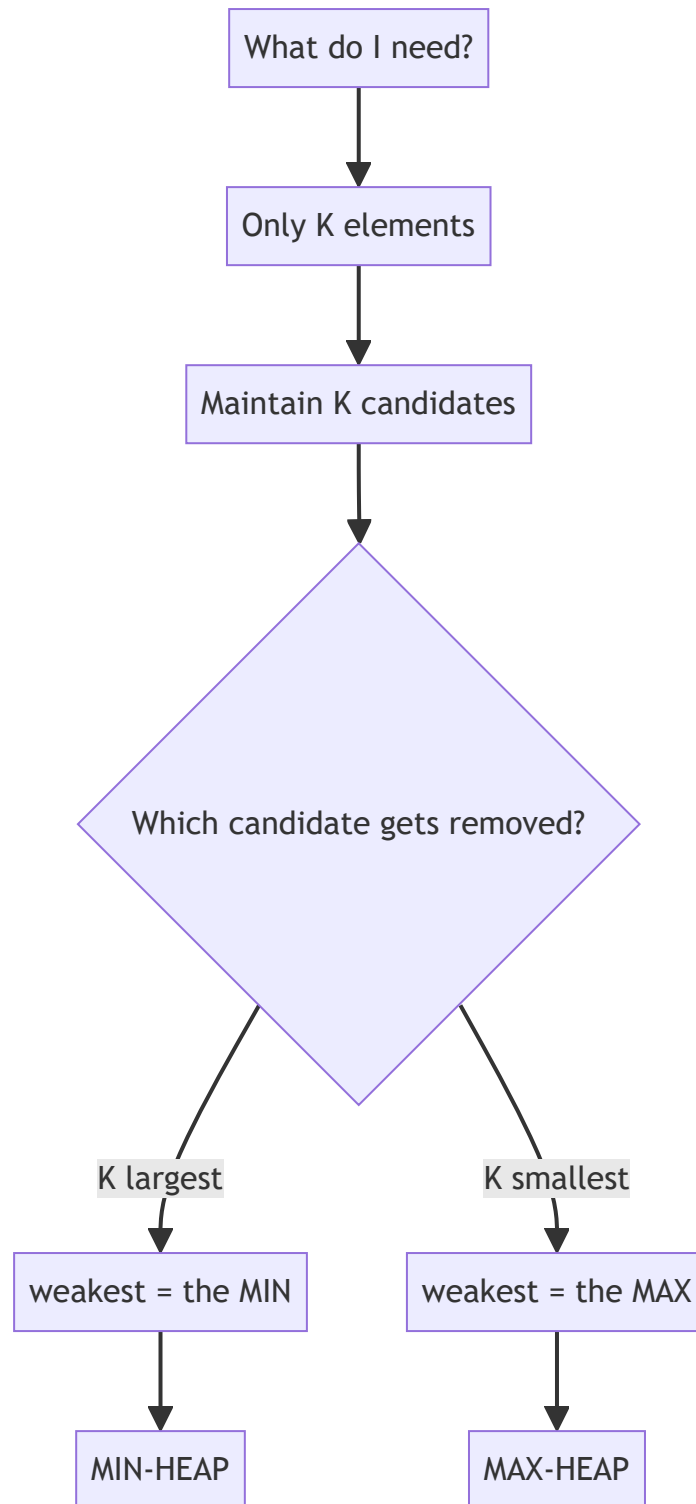
def find_kth_smallest(nums, k):
    heap = []                                # max-heap (negated) holding the K smallest
    for x in nums:
        heapq.heappush(heap, -x)
        if len(heap) > k:
            heapq.heappop(heap)              # evict the weakest (largest)
    return -heap[0]
```

The optimised variant (compare with the root first, then `heapreplace`):

```
import heapq

def find_kth_largest_optimized(nums, k):
    heap = []
    for x in nums:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:          # only enter if stronger than the weakest
            heapq.heapreplace(heap, x)  # pop + push in one O(log k) step
    return heap[0]
```

Full pattern.



2.7 Top K Frequent Elements (LeetCode 347)

`nums = [1, 1, 1, 2, 2, 3]` , `k = 2` → the 2 most frequent elements: `[1, 2]` .

What changes? For ordinary Top K we compare **values**. For Top K Frequent we compare **frequencies** — so we first need a hash map.

frequency map: $1 \rightarrow 3$ $2 \rightarrow 2$ $3 \rightarrow 1$
candidates: (element, frequency) = (1,3) (2,2) (3,1)

Apply the same reasoning: keep the K **highest** frequencies; the weakest candidate is the one with the **smallest frequency** → **min-heap** of (frequency, element) .

candidate	decision	min-heap of (element, frequency), weakest first
(1, freq 3)	heap not full: push	[(1, 3)]
(2, freq 2)	heap not full: push	[(2, 2), (1, 3)]
(3, freq 1)	weaker than the root: ignore	[(2, 2), (1, 3)]

If a new candidate (4, 4) then arrived, (2, 2) would be evicted, leaving {1, 4} — the same "evict the weakest" rule.

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    freq = Counter(nums)           # hash map: element -> count
    heap = []                     # min-heap of (count, element)
    for num, count in freq.items():
        heapq.heappush(heap, (count, num))
        if len(heap) > k:
            heapq.heappop(heap)    # evict the least frequent
    return [num for count, num in heap]
```

Complexity. Counting is $O(n)$; then each of the d distinct elements costs $O(\log k) \rightarrow O(n + d \log k)$, i.e. $O(n \log k)$ in the worst case. Space $O(d + k)$.

An important abstraction. You have moved beyond "*K largest numbers*". The real pattern is: **if I need the K best objects according to some score, maintain a heap of K objects ordered by that score.** The score can be a value, a frequency, a distance, a profit, an execution time, a priority...

2.8 K Closest Points to Origin (LeetCode 973)

Points (1,1), (2,3), (0,2), (5,1), (1,0), $K = 2 \rightarrow$ the 2 points closest to (0,0) .

Don't calculate everything yet — decide three things: the score, the K candidates, the heap.

What are we ranking by? → distance from the origin
What do we want? → the K SMALLEST distances
Weakest of the K candidates? → the LARGEST distance
Need fast access to largest → MAX-HEAP

You did not start from "K closest → max-heap"; you **derived** it. That is the skill.

Optimisation — skip the square root. `sqrt` preserves ordering: $d_1 < d_2 \Leftrightarrow d_1^2 < d_2^2$. So rank by $x^2 + y^2$ — same order, less work.

point	$\text{dist}^2 = x^2 + y^2$	action	max-heap contents (point:dist ²)
(1,1)	2	push	[(1,1):2]
(2,3)	13	push	[(2,3):13, (1,1):2]
(0,2)	4	push, then evict farthest (2,3) with dist ² 13	[(0,2):4, (1,1):2]
(5,1)	26	push, then evict farthest (5,1) with dist ² 26	[(0,2):4, (1,1):2]
(1,0)	1	push, then evict farthest (0,2) with dist ² 4	[(1,1):2, (1,0):1]

Result: (1,1) and (1,0).

```
import heapq

def k_closest(points, k):
    heap = []                                # max-heap on distance^2
    for x, y in points:
        dist = x * x + y * y                # no sqrt needed: it preserves order
        heapq.heappush(heap, (-dist, x, y))
        if len(heap) > k:
            heapq.heappop(heap)              # evict the farthest
    return [(x, y) for _, x, y in heap]
```

2.9 Pattern 1 summary

Problem	What is the "score"?	Want	Heap
K largest	value	largest K	Min-heap
K smallest	value	smallest K	Max-heap
K frequent	frequency	largest K	Min-heap
K closest	distance	smallest K	Max-heap

Mental model. Keep only the K candidates that can still become part of the answer. The heap exposes the weakest candidate so we can eject it when a better candidate arrives. **The heap direction is determined by the candidate you need to eliminate.**

2.10 Harder problems for Pattern 1

1A. Find K Pairs with Smallest Sums (LeetCode 373)

`nums1 = [1,7,11]`, `nums2 = [2,4,6]`, `k = 3` → the 3 pairs (u, v) with the smallest $u + v$. Both arrays are sorted.

Why it is harder. The naive approach considers every pair ($3 \times 3 = 9$ here, $m \times n$ in general). But both arrays are sorted, so there is structure. For a fixed `nums1[i]` the sums **increase** as we walk through `nums2`:

```
row of 1 : 1+2, 1+4, 1+6 ← already sorted
row of 7 : 7+2, 7+4, 7+6 ← already sorted
row of 11: 11+2, 11+4, 11+6 ← already sorted
```

Each row is a **sorted stream of candidates**, and we want the globally smallest ones — that is a **K-way merge** in disguise.

Thought process. We don't need all pairs. Put the **first** candidate of each row into a min-heap: `(1+2)`, `(7+2)`, `(11+2)`. Take the smallest; if it was `(1, 2)`, its row's next candidate is `(1, 4)` — push that. Stop after `k` pairs.

popped	pushed (next in that row)	heap after
pair (1, 2), sum 3	(1+4=5)	[1+4=5, 7+2=9, 11+2=13]
pair (1, 4), sum 5	(1+6=7)	[1+6=7, 7+2=9, 11+2=13]
pair (1, 6), sum 7	-	[7+2=9, 11+2=13]

```
import heapq

def k_smallest_pairs(nums1, nums2, k):
    if not nums1 or not nums2 or k == 0:
        return []
    heap = []
    for i in range(min(k, len(nums1))):
        heapq.heappush(heap, (nums1[i] + nums2[0], i, 0))
    result = []
    while heap and len(result) < k:
        _, i, j = heapq.heappop(heap)
        result.append([nums1[i], nums2[j]])
        if j + 1 < len(nums2):
            heapq.heappush(heap, (nums1[i] + nums2[j + 1], i, j + 1))
    return result
```

Complexity. The heap never holds more than `min(k, len(nums1))` entries → **$O(k \log k)$** time, **$O(k)$** space (plus the output).

Key insight. This looks like Top K, but the deeper pattern is: **top-K candidates generated from sorted candidate streams → heap.**

1B. Kth Smallest Element in a Sorted Matrix (LeetCode 378)

Matrix (each row sorted): `[[1, 5, 9], [10, 11, 13], [12, 13, 15]]`, `k = 8` → **13**.

Thought process. Each row is sorted, so again we have multiple sorted streams. Put the first element of every row into a min-heap; extract `1`, insert `5`; extract `5`, insert `9`; ... continue until `k` elements have been extracted. The last one extracted is the `k`-th smallest.

k-th pop	value popped	pushed	heap (values)
1	1 (row 0)	5	[5, 10, 12]
2	5 (row 0)	9	[9, 10, 12]
3	9 (row 0)	-	[10, 12]
4	10 (row 1)	11	[11, 12]
5	11 (row 1)	13	[12, 13]
6	12 (row 2)	13	[13, 13]
7	13 (row 1)	-	[13]
8	13 (row 2)	15	[15]

The 8th smallest = **13**

```
import heapq

def kth_smallest_matrix(matrix, k):
    heap = []
    for r in range(len(matrix)):
        heapq.heappush(heap, (matrix[r][0], r, 0))
    for _ in range(k):
        value, r, c = heapq.heappop(heap)
        if c + 1 < len(matrix[r]):
            heapq.heappush(heap, (matrix[r][c + 1], r, c + 1))
    return value
```

Complexity. For an $n \times n$ matrix the heap holds $O(n)$ entries $\rightarrow O(k \log n)$ time, $O(n)$ space.

Pattern connection. This one technically overlaps with K-way merge (Part 5) — *that is intentional*. Real problems don't announce which pattern they belong to. (Other approaches exist for this particular problem, such as binary search on the value range, but the heap solution is the one that generalises.)

Part 3 · Pattern 2 — Two Heaps

Recognition. "A dynamic median" / "a boundary between the smaller half and the larger half that keeps moving as data arrives."

This time nobody told us to use a heap. Let's reason from the operation we need.

3.1 Deriving it: Find Median from Data Stream (LeetCode 295)

Problem. You are given n numbers and must report the **median after every insertion**. Support potentially **millions** of insertions.

insert 5 → median = 5 insert 2 → median = 3.5 insert 10 → median = 5 insert 7 → median = 6

Step 1 — forget the word "heap". We have a growing set:

5 → 5, 2 → 5, 2, 10 → 5, 2, 10, 7 → ...

After every insertion we need the median. For [2, 5, 7, 10] the median sits in the **middle region**:

2 5 | 7 10
 ↑
middle region

We do **not** need the whole set sorted.

Step 2 — what information is sufficient? Think of the numbers smaller than the median and the numbers larger than the median. *What if we maintained those two groups separately — what would we need from each?*

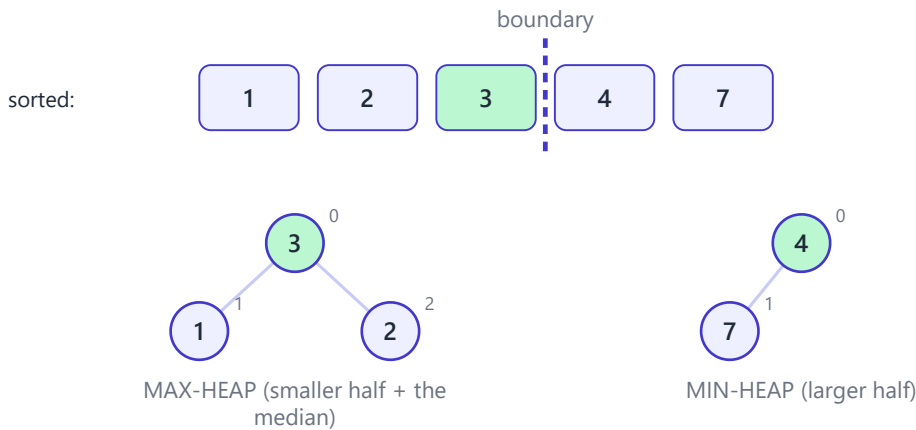
Answer: the largest element of the left (smaller) half and the smallest element of the right (larger) half. Those two elements sit right around the middle.

smaller half | larger half
 ↑
 boundary

Step 3 — which data structures?

Half	What we need from it	Heap
Left / smaller half	its largest element at the root	MAX-heap
Right / larger half	its smallest element at the root	MIN-heap

Two heaps = the two halves of the sorted order; their roots sit on either side of the median



5 numbers (odd): median = max-heap root = 3; even count: median = (max-heap root + min-heap root) / 2

Invariant: $\text{len}(\text{max-heap}) == \text{len}(\text{min-heap})$ or $\text{len}(\text{max-heap}) == \text{len}(\text{min-heap}) + 1$; every value in the max-heap $\leq$ every value in the min-heap

maxHeap		minHeap
↓		↓
largest on left		smallest on right

3.2 The size invariant (and the "why only the max-heap gets the extra?" doubt)

If the left side has one extra element the median is simply its root:

maxHeap = [1, 3, 5]	minHeap = [7, 9]	→ median = 5
maxHeap = [1, 3]	minHeap = [5, 7]	→ median = (3 + 5) / 2 = 4

So we aim for this invariant:

size(maxHeap) == size(minHeap)	(even count)
OR	
size(maxHeap) == size(minHeap) + 1	(odd count)

A doubt worth answering: why should the *max-heap* hold the extra element — why not the *min-heap*? It is a **design choice, not a mathematical requirement**. With the max-heap holding the extra element, for an odd number of elements the median sits directly at the **root of the max-heap** — no averaging needed:

5 elements: [1, 2] | [4, 7] with the middle value 3 placed on the left maxHeap = [3, 1, 2]
minHeap = [4, 7] median = maxHeap.top() = 3

You could equally let the min-heap hold the extra (maxHeap = [1, 2] , minHeap = [3, 4, 7] , median = minHeap.top()). **The real invariant is: the two heaps differ in size by at most 1** — pick one side and be consistent. The common convention gives the extra element to the max-heap (the left side).

3.3 Where does a new number go?

Suppose maxHeap.top = 5 , minHeap.top = 8 and a new number 6 arrives. It belongs relative to the boundary 5 | 8 . **Rule:**

```
if x <= maxHeap.top():      # belongs to the smaller half
    put x in maxHeap
else:
    put x in minHeap
```

Then **rebalance** if the size difference exceeds 1 — in whichever direction is necessary:

- if the **max-heap has 2 more** → move maxHeap.top() to the min-heap;
- if the **min-heap is bigger** → move minHeap.top() to the max-heap.

(With the "max-heap may have one extra" convention the min-heap is never allowed to exceed the max-heap, so the check after insertion is len(right) > len(left) .)

Rebalancing example — insert 2 :

step	max-heap (left)	min-heap (right)	sizes
start	[1, 3, 5]	[7, 8]	3 vs 2 (ok)
insert 2: 2 <= 5, so it goes LEFT	[1, 2, 3, 5]	[7, 8]	4 vs 2 (difference 2!)
rebalance: move max-heap root 5 to the right	[1, 2, 3]	[5, 7, 8]	3 vs 3 (ok)

3.4 The complete invariant, in your own words

Max-heap holds the smaller half; min-heap holds the larger half. Their roots meet at the median, and balancing keeps the halves almost equal.

1. every value in the max-heap is <= every value in the min-heap;
2. size(max) == size(min) or size(max) == size(min) + 1 .

(A common slip when first stating it: "if they differ by more, move the max-heap top to the min-heap" — that is only one of the two directions. Rebalance whichever way is necessary.)

3.5 Dry run and code

Stream 5, 2, 10, 7:

insert	goes to	rebalance	max-heap (smaller half)	min-heap (larger half)	median
5	max-heap (empty)	-	[5]	[]	5
2	max-heap ($x \leq$ its root)	move 5: max-heap $\rightarrow$ min-heap	[2]	[5]	3.5
10	min-heap ($x >$ max-heap root)	move 5: min-heap $\rightarrow$ max-heap	[2, 5]	[10]	5
7	min-heap ($x >$ max-heap root)	-	[2, 5]	[7, 10]	6.0

The medians are 5, 3.5, 5, 6 — exactly the expected output.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.left = []    # max-heap (stored negated): the smaller half
        self.right = []   # min-heap: the larger half

    def add_num(self, x):
        if not self.left or x <= -self.left[0]:
            heapq.heappush(self.left, -x)
        else:
            heapq.heappush(self.right, x)
        # rebalance so that len(left) == len(right) or len(left) == len(right) + 1
        if len(self.left) > len(self.right) + 1:
            heapq.heappush(self.right, -heapq.heappop(self.left))
        elif len(self.right) > len(self.left):
            heapq.heappush(self.left, -heapq.heappop(self.right))

    def find_median(self):
        if len(self.left) > len(self.right):
            return -self.left[0]
        return (-self.left[0] + self.right[0]) / 2
```

Complexity.

Operation	Cost
add a number	$O(\log n)$ (one push, at most one pop + push)
get the median	$O(1)$ (read the roots)
space	$O(n)$

Mental model. Two heaps maintain the two sides of the sorted order; their roots are the two elements surrounding the median.

3.6 Harder problem — 2A. Sliding Window Median (LeetCode 480)

`nums = [1, 3, -1, -3, 5, 3, 6, 7]` , `k = 3` → the median of every window of size 3.

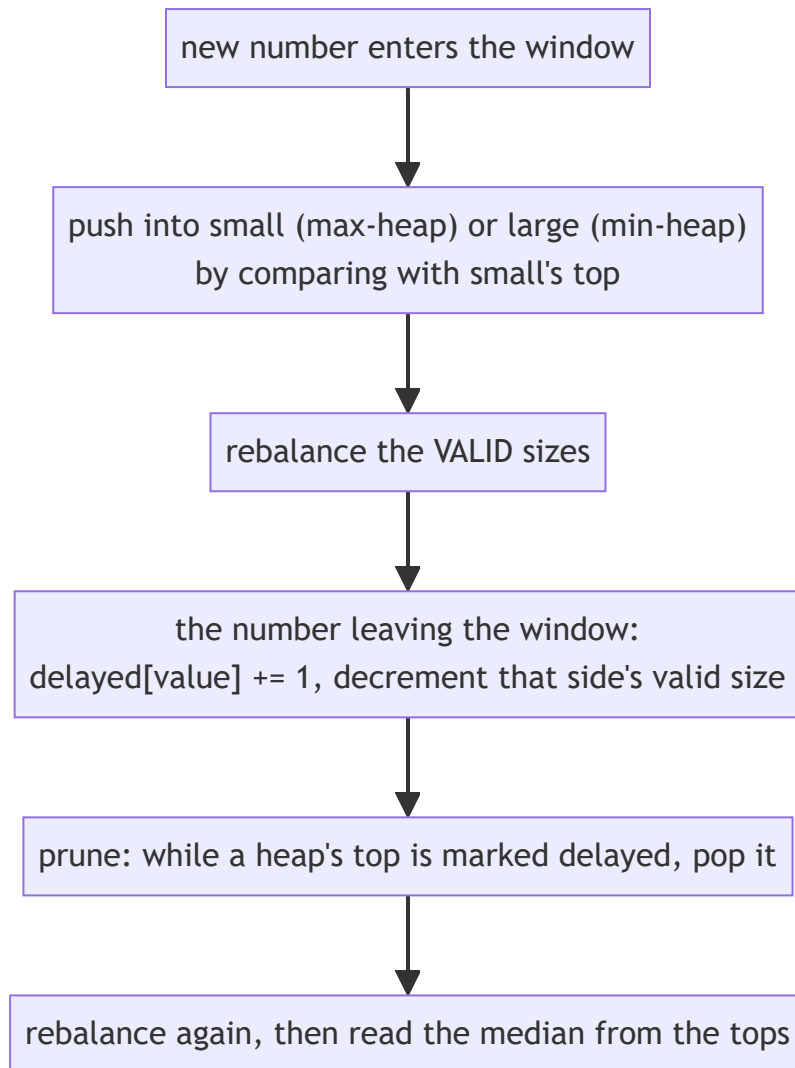
window start	window	sorted (for checking)	median
0	[1, 3, -1]	[-1, 1, 3]	1.0
1	[3, -1, -3]	[-3, -1, 3]	-1.0
2	[-1, -3, 5]	[-3, -1, 5]	-1.0
3	[-3, 5, 3]	[-3, 3, 5]	3.0
4	[5, 3, 6]	[3, 5, 6]	5.0
5	[3, 6, 7]	[3, 6, 7]	6.0

Why two heaps? Each window needs *smaller half* | *larger half* → **max-heap** | **min-heap**. But unlike the plain median finder, **elements leave the window**. A heap cannot efficiently remove an arbitrary element, so this creates the *stale-entry problem*. This problem combines:

Two heaps + lazy deletion

Core idea. When an element leaves the window we do **not** search for it inside a heap. We **record that it must be deleted** and remember how many *valid* elements each heap really holds. Whenever a stale value reaches a heap's top, we pop it away. Sizes are tracked separately (`small_size` , `large_size`) because a heap's `len()` includes stale entries.

The algorithm, per step:



Correction. The version of this solution in the original notes gives the right answer for the example above, but it is **not reliable**: on `nums = [-16, 11, 21, -13]`, `k = 2` it crashes with an `IndexError`, and against a brute-force median on 600 random arrays it returned a wrong answer or crashed on **196 of them** (it fails on roughly a quarter of random arrays even when every value is distinct). The cause is bookkeeping: it records deletions **by index** but decides which heap the outgoing element lives in by **comparing values**, so the two size counters and the heap tops can fall out of sync. The version below records deletions **by value** (the standard, safe formulation) and passed **600/600** random tests against a brute-force median.

```

import heapq
from collections import defaultdict

def median_sliding_window(nums, k):
    small, large = [], []          # small: max-heap (negated), large: min-heap
    delayed = defaultdict(int)     # value -> how many copies are waiting to be deleted
    small_size = large_size = 0    # number of *valid* elements in each heap

    def prune(heap, is_small):
        # drop stale entries that have surfaced at the top
        while heap:
            val = -heap[0] if is_small else heap[0]
            if delayed[val] > 0:
                delayed[val] -= 1
                heapq.heappop(heap)
            else:
                break

    def rebalance():
        nonlocal small_size, large_size
        if small_size > large_size + 1:
            heapq.heappush(large, -heapq.heappop(small))
            small_size -= 1
            large_size += 1
            prune(small, True)
        elif small_size < large_size:
            heapq.heappush(small, -heapq.heappop(large))
            large_size -= 1
            small_size += 1
            prune(large, False)

    def add(x):
        nonlocal small_size, large_size
        if not small or x <= -small[0]:
            heapq.heappush(small, -x)
            small_size += 1
        else:
            heapq.heappush(large, x)
            large_size += 1
        rebalance()

    def remove(x):
        nonlocal small_size, large_size
        delayed[x] += 1          # lazy deletion: just remember it
        if x <= -small[0]:
            small_size -= 1
            if x == -small[0]:
                prune(small, True)
        else:

```

```

        large_size -= 1
        if large and x == large[0]:
            prune(large, False)
        rebalance()

def median():
    if small_size > large_size:
        return float(-small[0])
    return (-small[0] + large[0]) / 2

for x in nums[:k]:
    add(x)
result = [median()]
for i in range(k, len(nums)):
    add(nums[i])
    remove(nums[i - k])
    result.append(median())
return result

```

Complexity. Every element is pushed once and popped at most once, each operation $O(\log n) \rightarrow O(n \log n)$ time; $O(n)$ space (stale entries can linger until they surface).

Recognition. *Dynamic median + elements entering and leaving $\rightarrow$ two heaps + lazy deletion.*

Part 4 · Pattern 3 — Repeatedly Process the Best Available Element

Recognition. "At every step I need the current minimum/maximum, then I modify the set and repeat."

This is different from Top K. **Top K** maintains a *bounded* set of candidates. **Pattern 3** repeatedly *consumes* the best available element(s) and inserts a resulting element back.

4.1 Deriving it: Connect Ropes (LeetCode 1167 — Minimum Cost to Connect Sticks)

Problem. Ropes have lengths $[4, 3, 2, 6]$. Connecting two ropes costs $\text{length1} + \text{length2}$, and the result is one rope of that length. Connect all ropes into one, at **minimum total cost**.

Connect $2 + 3 = 5$ cost = 5 Connect $4 + 5 = 9$ cost = 9 Connect $6 + 9 = 15$ cost = 15 Total = 29

Don't think about heaps yet. Which two ropes should we combine at every step? Compare with combining $4 + 6 = 10$ first:

strategy	merges (cost)	total
two SMALLEST first	$2+3=5, 4+5=9, 6+9=15$	29
$4 + 6$ first	$4+6=10, 2+3=5, 5+10=15$	30

Common confusion. "Isn't the total cost the same regardless of the order?" — **No.** The *final rope* (length 15) is always the same, but the **total cost is the sum of every intermediate rope we create**, and that differs.

Why smallest-first? — the mathematical reason

When two ropes are combined, **the resulting rope's length is counted again** in any future combination it takes part in. So count how many times each **original** rope contributes to the cost:

Smallest-first: $(2+3) + (4+2+3) + (6+4+2+3) = 5 + 9 + 15 = 29$
 $4+6$ first: $(4+6) + (2+3) + (2+3+4+6) = 10 + 5 + 15 = 30$

rope	times it is counted (smallest-first)	contribution	times counted (4+6 first)	contribution
2	3	6	2	4
3	3	9	2	6
4	2	8	2	8
6	1	6	2	12
total		29		30

The principle. Every time a rope participates in another merge, its length contributes to the cost again. Therefore we want small lengths to participate in more merges and large lengths to participate in fewer merges. That leads to: **repeatedly combine the two smallest available ropes.**

(This is exactly **Huffman coding**: repeatedly merge the two least-frequent symbols. Same algorithm, same proof.)

The data-structure requirement

Need the two smallest → combine them → insert the new rope

```

      ↑                               |
      |                               |
      |_____ again need the two smallest ←|
  
```

What gives us the current smallest element efficiently? A **min-heap**.

"Why extract TWO elements each time, not one?" Because a connection always involves **two** ropes: rope A + rope B → new rope. To perform the merge we need the smallest **and** the second-smallest.

heap [2, 3, 4, 6]: extract 2, extract 3 → combine 5 → insert 5 → heap [4, 5, 6]
 extract 4, extract 5 → combine 9 → insert 9 → heap [6, 9] ...

extract two smallest	cost of this merge	running total	heap afterwards
2 + 3	5	5	[4, 5, 6]
4 + 5	9	14	[6, 9]
6 + 9	15	29	[15]

How many merges? (a slip worth fixing)

" $n/2$?" — **No.** Every merge takes 2 ropes and produces 1, so the number of ropes drops by **exactly 1** each time: $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ is **3 merges**. For n ropes there are **$n - 1$ merges**.

Complexity. Build the heap $O(n)$; $n - 1$ merges, each with 2 extracts + 1 insert at $O(\log n)$ → **$O(n \log n)$** time, **$O(n)$** space.

```
import heapq

def connect_ropes(ropes):
    ropes = list(ropes)
    heapq.heapify(ropes)          # O(n)
    total = 0
    while len(ropes) > 1:
        a = heapq.heappop(ropes)  # smallest
        b = heapq.heappop(ropes)  # second smallest
        cost = a + b
        total += cost
        heapq.heappush(ropes, cost) # the merged rope is still available
    return total
```

Note: `heapq.heapify` works in place. The copy `ropes = list(ropes)` at the top keeps the caller's list intact.

4.2 Same pattern, opposite direction: Last Stone Weight (LeetCode 1046)

Problem. Stones `[2, 7, 4, 1, 8, 1]`. Repeatedly take the **two heaviest** stones, $x \geq y$. If $x == y$ both are destroyed; else the stone $x - y$ remains. Return the last stone's weight (0 if none).

What operation do we repeatedly need, and which heap makes it cheap?

Need the two LARGEST → repeatedly need the current maximum → MAX-HEAP

The first operation must involve `8` and `7`. After they collide, `8 - 7 = 1` is a **new** stone, so it is **inserted back**.

two heaviest	result	heap afterwards (largest first)
8, 7	$8 - 7 = 1$, push back	[4, 2, 1, 1, 1]
4, 2	$4 - 2 = 2$, push back	[2, 1, 1, 1]
2, 1	$2 - 1 = 1$, push back	[1, 1, 1]
1, 1	$1 == 1$: both destroyed	[1]

Final stone weight = **1**

Unlike Connect Ropes we do not *always* insert a result — a pair of equal stones simply disappears.

Connect Ropes: need minimum → extract → combine → insert result → repeat
 Last Stone Weight: need maximum → extract → process → (maybe) insert result → repeat

The heap invariant

"By invariant, what do you mean?" A condition we keep true throughout the algorithm — the rule that says *what the data structure currently represents*.

- Top K largest: *"the min-heap contains the K largest elements seen so far."*
- Last Stone Weight: **"the max-heap contains exactly all stones that are still remaining."**

Every time we extract the two largest, process them and insert the difference (if any), we **restore** that invariant. It answers *"should the new stone be pushed back?"* — yes, because it is a remaining stone.

```
import heapq

def last_stone_weight(stones):
    heap = [-x for x in stones]          # max-heap via negation
    heapq.heapify(heap)
    while len(heap) > 1:
        x = -heapq.heappop(heap)         # heaviest
        y = -heapq.heappop(heap)         # second heaviest
        if x != y:
            heapq.heappush(heap, -(x - y))
    return -heap[0] if heap else 0
```

Complexity. Each stone is removed/inserted at most once per collision → **$O(n \log n)$** time, **$O(n)$** space.

Generalised recognition. When each step requires the current best available element, and processing it changes the remaining set, consider a heap.

4.3 More variations

(a) Repeatedly take the smallest, modify it, put it back. [5, 3, 8, 2, 7] — which heap, what invariant? **Min-heap.** Invariant: *the heap always contains exactly the remaining elements*. (The invariant describes *what the heap represents*; the heap **type** tells us how we access it efficiently.)

(b) Shortest job first. A processor always picks the shortest remaining task and finishes it before choosing again — find the order. Recognise it without the word "heap": *"I repeatedly need the smallest among a changing set → I need efficient access to the current extreme → min-heap"*.

```
import heapq

def shortest_job_order(times):
    heap = [(t, i) for i, t in enumerate(times)]
    heapq.heapify(heap)
    order = []
    while heap:
        t, i = heapq.heappop(heap)
        order.append(i)
    return order
```

(c) Double the smallest. [10, 4, 7, 2, 9] — take the smallest, multiply by 2, put it back, repeat 3 times. Each iteration is: *peek/extract minimum* → *modify* → *push the modified value back* (the modified value is still part of the remaining set).

extract smallest	push back doubled	heap afterwards
2	4	[4, 4, 7, 9, 10]
4	8	[4, 7, 8, 9, 10]
4	8	[7, 8, 8, 9, 10]

```
import heapq

def double_smallest(nums, times):
    heap = list(nums)
    heapq.heapify(heap)
    for _ in range(times):
        x = heapq.heappop(heap)          # smallest
        heapq.heappush(heap, x * 2)     # modified value goes back
    return sorted(heap)
```

Why not just sort once?

"Sort [2, 4, 7, 9, 10] once and take from the front?" — If you only remove elements in increasing order, sorting is perfectly reasonable: $O(n \log n)$. But the doubled values must be **inserted back into the right place**. A heap is more natural when the set **keeps changing while you repeatedly need its min/max**: inserting a new value costs $O(\log n)$, whereas keeping a sorted array costs $O(n)$ per insertion.

So the real distinction is not "heap is always better than sorting" but:

"Does the set keep changing while I repeatedly need its minimum/maximum?" If yes → heap.

Pattern 3 checkpoint

Situation	Heap
Repeatedly need the smallest	Min-heap
Repeatedly need the largest	Max-heap
Process it and remove it	Extract
Processing creates a new candidate	Push it back
What does the heap represent?	The current remaining elements

Mental model. *Repeatedly need the current smallest/largest, process it, and modify the remaining set* → heap.

4.4 Harder problems for Pattern 3

3A. Minimum Cost to Hire K Workers (LeetCode 857)

Each worker has `quality[i]` and `wage[i]` (minimum expected wage). Hire exactly `k` workers. Everyone in the group is paid **in proportion to quality** (the same wage/quality ratio for all), and nobody may be paid less than their minimum wage. Minimise the total pay. Example: `quality = [10, 20, 5]`, `wage = [70, 50, 30]`, `k = 2` → **105**.

The important transformation. For a chosen group, every worker is paid at the **same rate** `r` per unit of quality, and that rate must satisfy every member: `r >= wage[i] / quality[i]` for all of them. So the group's rate is the **largest** ratio in the group:

```
payment = max_ratio × total_quality
```

So sort workers by `wage / quality`. Scanning from the smallest ratio upward, **the current worker's ratio is the maximum ratio of any group made from the workers seen so far**. Now, for that fixed rate, we want the **K smallest total quality** among the workers seen so far.

```
sort by ratio
  ↓
current ratio fixes the rate (it is the maximum in the group)
  ↓
among workers seen so far, keep the K smallest qualities
  ↓
"keep the K smallest" → MAX-heap of quality
```

Why a **max**-heap? If we ever hold more than `K` workers, we **remove the one with the largest quality** (the weakest candidate for "K smallest").

worker (sorted by ratio)	wage/quality	group qualities (max-heap)	sum of qualities	cost = ratio x sum
q=20, w=50	2.50	[20]	20	-
q=5, w=30	6.00	[5, 20]	25	150
q=10, w=70	7.00	[5, 10]	15	105

Minimum cost = **105**

```

import heapq

def min_cost_to_hire(quality, wage, k):
    workers = sorted(zip(quality, wage), key=lambda x: x[1] / x[0]) # by wage/quality ratio
    max_heap = [] # negated qualities of the current group
    quality_sum = 0
    answer = float('inf')
    for q, w in workers:
        heapq.heappush(max_heap, -q)
        quality_sum += q
        if len(max_heap) > k:
            quality_sum += heapq.heappop(max_heap) # popped value is negative: removes the largest quality
        if len(max_heap) == k:
            answer = min(answer, (w / q) * quality_sum)
    return answer

```

Complexity. Sort $O(n \log n)$ + heap $O(n \log k) \rightarrow O(n \log n)$.

Why this is a difficult heap problem. The heap isn't obvious from the statement — you must first *derive* that the ratio ordering makes the payment formula linear. It is a **heap + sorting + greedy** combination.

3B. Furthest Building You Can Reach (LeetCode 1642)

heights = [4,2,7,6,9,14,12], bricks = 5, ladders = 1. Moving to a taller building costs (height difference) bricks **or** one ladder (a ladder covers any climb, of any size). Moving to a lower or equal building is free. What is the index of the furthest building you can reach? $\rightarrow$ 4.

Key observation. We don't know in advance which climbs should get the ladders. But **obviously the ladders should end up covering the *largest* climbs** — a ladder saves the most bricks when used on the biggest climb.

So process buildings left to right, and treat the ladders as a **budget of "free passes" that we keep flexible**: at every moment the `ladders` biggest climbs seen so far hold the ladders, and every other climb is paid with bricks.

A min-heap of size `ladders` holds the climbs that currently own a ladder. The root is the *smallest* of them — the first to give up its ladder when a bigger climb appears.

```

new climb c → push c into the heap (it tentatively gets a ladder)
if the heap now holds more than 'ladders' climbs:
    pop the SMALLEST one – that climb must pay with bricks
if bricks < 0: we are stuck – return the current index

```

step	climb	what happens	ladder heap (min-heap)	bricks left
4 -> 2	-2	free (no climb)	[]	5
2 -> 7	5	climb 5: give it a ladder	[5]	5
7 -> 6	-1	free (no climb)	[5]	5
6 -> 9	3	ladders full: smallest climb 3 pays with bricks	[5]	2
9 -> 14	5	ladders full: smallest climb 5 pays with bricks	[5]	-3

step	climb	what happens	ladder heap (min-heap)	bricks left
		bricks < 0: cannot reach building 5		-3

Furthest building index = **4**

```
import heapq

def furthest_building(heights, bricks, ladders):
    """Min-heap of size <= ladders keeps the LARGEST climbs (they get the ladders);
    every climb pushed out of the heap has to be paid for with bricks."""
    heap = []
    for i in range(len(heights) - 1):
        climb = heights[i + 1] - heights[i]
        if climb <= 0:
            continue
        heapq.heappush(heap, climb)
        if len(heap) > ladders:
            bricks -= heapq.heappop(heap)      # smallest climb among the ladder-holders -> bricks
        if bricks < 0:
            return i
    return len(heights) - 1
```

An **equivalent view** (spend bricks on everything; when bricks run out, convert the *largest* climb so far into a ladder and get its bricks back) uses a **max-heap**:

```
import heapq

def furthest_building_maxheap(heights, bricks, ladders):
    """Equivalent view: spend bricks on everything, and when bricks run out
    convert the LARGEST climb so far into a ladder (refund its bricks)."""
    heap = []                                     # max-heap (negated) of climbs paid with bricks
    for i in range(len(heights) - 1):
        climb = heights[i + 1] - heights[i]
        if climb <= 0:
            continue
        heapq.heappush(heap, -climb)
        bricks -= climb
        if bricks < 0:
            if ladders == 0:
                return i
            ladders -= 1
            bricks += -heapq.heappop(heap)      # refund the largest climb
    return len(heights) - 1
```

Correction. The original notes gave the following code, but it mixes the two views — it pushes every climb into a **min**-heap, spends bricks on every climb, and when bricks run out it refunds the **smallest** climb:

```
python heapq.heappush(heap, climb); bricks -= climb if bricks < 0: ... smallest =  
heapq.heappop(heap); bricks += smallest # <-- wrong climb gets the ladder
```

It happens to return the right answer for the example above, but it gives the ladder to the *smallest* climb instead of the *largest*. On `heights = [1,5,1,2,3,4,10000]`, `bricks = 4`, `ladders = 1` it returns **3** instead of **5**, and it disagreed with a brute-force answer on 29 of 1500 random cases.

approach	result	explanation
source's min-heap-pop-smallest code	3	spends the ladder on the tiny climb 1 and burns bricks on the big climb 4
corrected code (either version)	5	ladder goes to the biggest climb (4) so bricks last longer

Why "pop the smallest" gives the ladder to the largest climb. Suppose the climbs met so far are `2, 5, 7` and there is one ladder (heap size limit 1):

```
climb 2 → heap [2] (2 holds the ladder)  
climb 5 → push 5 → [2, 5] too many → pop 2 → 2 pays with bricks → heap [5]  
climb 7 → push 7 → [5, 7] too many → pop 5 → 5 pays with bricks → heap [7]
```

The ladder finishes on `7` (the largest) and bricks pay for `2 + 5`. **Whenever a bigger climb shows up, the ladder moves to it, and the displaced smaller climb pays with bricks.**

Complexity. $O(n \log L)$ where `L = ladders` (the heap never exceeds `L + 1` entries), $O(L)$ space.

Recognition. *Process sequentially, keep previous decisions flexible, and when resources run out, replace the least expensive previous allocation.* — a very important **greedy + heap** pattern.

Part 5 · Pattern 4 — K-Way Merge

Recognition. *K sorted arrays/lists/streams + "merge them" or "find the next smallest among their current elements".*

5.1 Deriving it: Merge K Sorted Arrays

You have 3 already-sorted arrays: $A = [1, 4, 9]$ $B = [2, 3, 10]$ $C = [5, 6, 8]$ Produce one sorted array containing all elements.

Before thinking about heaps — what is the problem we notice at each step? We have **K sorted sequences**, and we need to consider their **current elements simultaneously** (generally K arrays, not just three).

Suppose the current pointers are:

$A \rightarrow 1$ $B \rightarrow 2$ $C \rightarrow 5$

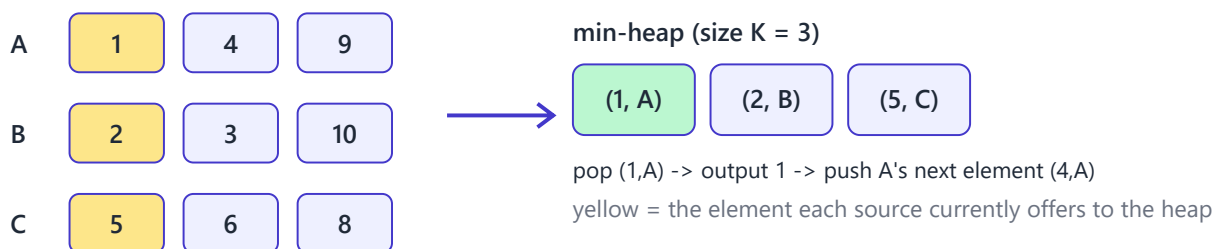
To build the merged array, **which one should we take next?** The **smallest current element** (1). After taking it, that array's pointer advances:

$A \rightarrow 4$ $B \rightarrow 2$ $C \rightarrow 5$ $\rightarrow$ now the smallest of (4, 2, 5) is 2 ...

So at every step we need: **the minimum among the current elements of K sorted arrays.**

How could a min-heap help? What goes in it? The heap contains **the current (front) element from each array** — *one candidate per source*, not every element:

K-way merge: the heap holds ONE current element per sorted source

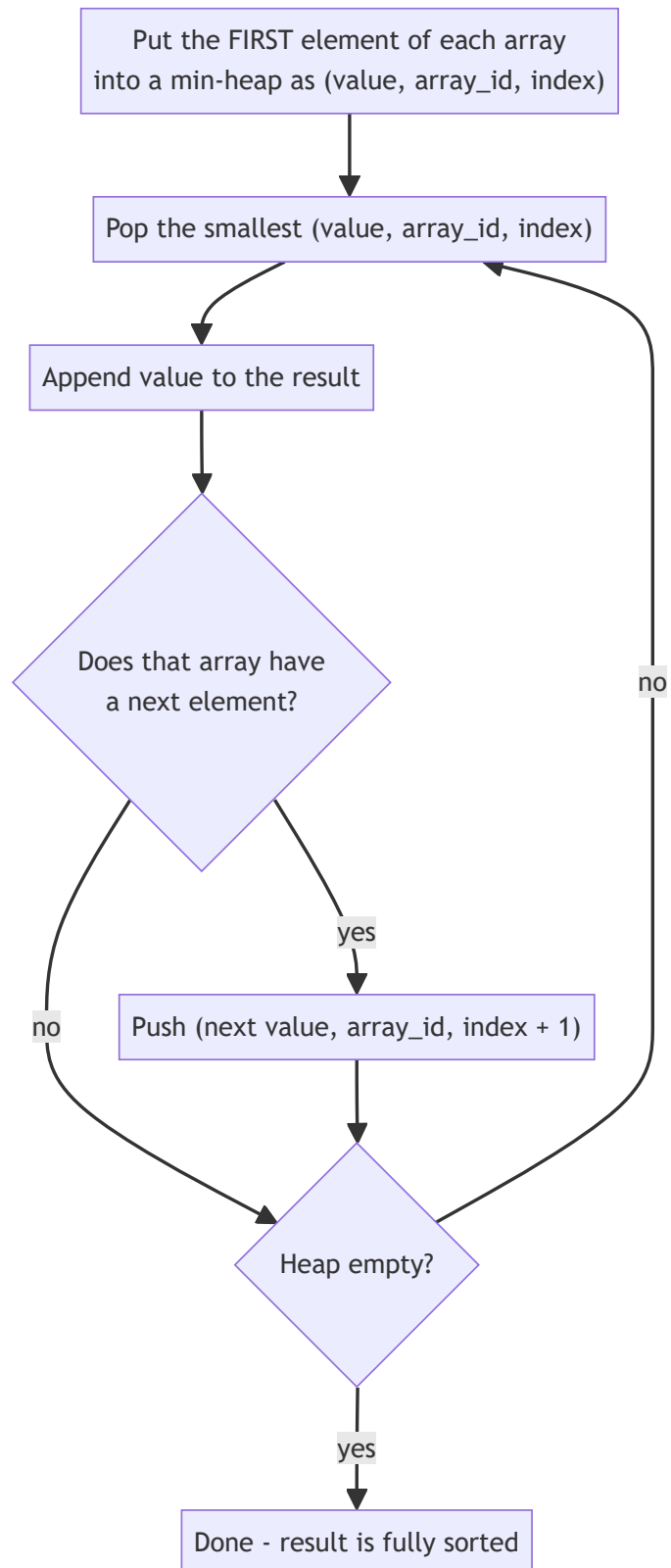


The heap hands us the smallest current element in $O(\log K)$. But there is one important detail: when we extract 1, **we must know which array it came from**, because only that array should advance. So a heap entry needs:

(value, array_id, index) e.g. (1, A, 0) (2, B, 0) (5, C, 0)

What do we do with 4, A's next element? Put it back into the heap.

The complete K-way merge loop



1. Put the first element of each array into a min-heap.
2. Extract the smallest (value, array_id, index).
3. Add value to the result.
4. Advance that array.
5. If that array still has elements, push its next element into the heap.

6. Repeat until the heap is empty.

step	pushed (that source's next)	output so far	heap (one per source)
start	-	-	[(1,A), (2,B), (5,C)]
pop (1,A)	(4,A)	[1]	[(2,B), (4,A), (5,C)]
pop (2,B)	(3,B)	[1, 2]	[(3,B), (4,A), (5,C)]
pop (3,B)	(10,B)	[1, 2, 3]	[(4,A), (5,C), (10,B)]
pop (4,A)	(9,A)	[1, 2, 3, 4]	[(5,C), (9,A), (10,B)]
pop (5,C)	(6,C)	[1, 2, 3, 4, 5]	[(6,C), (9,A), (10,B)]
pop (6,C)	(8,C)	[1, 2, 3, 4, 5, 6]	[(8,C), (9,A), (10,B)]
pop (8,C)	-	[1, 2, 3, 4, 5, 6, 8]	[(9,A), (10,B)]
pop (9,A)	-	[1, 2, 3, 4, 5, 6, 8, 9]	[(10,B)]
pop (10,B)	-	[1, 2, 3, 4, 5, 6, 8, 9, 10]	[]

```
import heapq

def merge_k_sorted(arrays):
    heap = []
    result = []
    for i, arr in enumerate(arrays):
        if arr:
            heapq.heappush(heap, (arr[0], i, 0))    # (value, which array, index in it)
    while heap:
        value, array_id, index = heapq.heappop(heap)
        result.append(value)
        next_index = index + 1
        if next_index < len(arrays[array_id]):
            heapq.heappush(heap, (arrays[array_id][next_index], array_id, next_index))
    return result
```

Why a heap and not repeated scanning?

At every step we need the minimum among K candidates:

Method	Cost per output element	Total for N elements
Scan all K current elements	$O(K)$	$O(N \cdot K)$
Min-heap of K candidates	$O(\log K)$	$O(N \log K)$

With N total elements and K arrays:

- **Time: $O(N \log K)$** — *not* $O(N \log N)$, because the heap only ever contains at most K candidates.
- **Space: $O(K)$** for the heap (plus the output).

(Python's `heapq.merge(*arrays)` does exactly this, lazily.)

Mental model. *One candidate from every sorted source → heap → take the global minimum → advance that source.*

5.2 Test without the word "heap": 100 sorted files

You have 100 sorted files, each containing millions of timestamps. Produce one **globally sorted stream** without loading all files into memory. *What would you maintain?*

A min-heap holding the next unread timestamp from each file, plus which file it belongs to.

```
Heap: (timestamp, file_id, pointer)
File 1 → 10    File 2 → 3    File 3 → 7    ...
The heap yields 3 → consume it from File 2 → advance File 2's pointer → push File 2's next
timestamp.
```

Memory use is $O(K)$ — one entry per file — no matter how large the files are. (This is *external sorting's* merge phase and the way sorted logs/streams are merged.)

One subtle point. You don't strictly need to store the whole pointer if the heap entry itself identifies the next item; conceptually "one current candidate per file" is exactly right.

Recognition. *K sorted sequences → maintain one current candidate from each → repeatedly take the smallest → advance that sequence → insert its next candidate.* The same pattern appears in **merging K sorted linked lists**, external sorting and merging sorted streams.

5.3 Harder problems for Pattern 4

4B. Merge K Sorted Linked Lists (LeetCode 23)

Same K-way principle, but now the "next element" is a **linked-list node**.

```
import heapq

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node)) # i breaks ties: ListNode is not comparable
    dummy = ListNode(0)
    tail = dummy
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Why is `i` in the tuple? `(node.val, i, node)` — if two nodes have the **same value**, Python compares the *next* tuple element, and a `ListNode` is not comparable, which would raise `TypeError`. The list index `i` is a unique tie-breaker so the comparison never reaches the node.

`ListNode` here is the usual `class ListNode: def __init__(self, val=0, next=None):`.

Complexity. N total nodes, K lists $\rightarrow O(N \log K)$ time, $O(K)$ extra space (nodes are re-linked, not copied).

4A. Smallest Range Covering Elements from K Lists (LeetCode 632)

Lists: `[4,10,15,24,26]`, `[0,9,12,20]`, `[5,18,22,30]`. Find the **smallest range** `[a, b]` containing **at least one element from every list**. $\rightarrow [20, 24]$.

Key observation. We need **one candidate from every list** — exactly K-way-merge territory. Keep one pointer per list. At any moment the pointers define:

```
current minimum = smallest of the K current values    (the heap root)
current maximum = largest of the K current values    (tracked separately)
current range    = [min, max]        width = max - min
```

How can the range improve? If the minimum is `x`, keeping it fixed the range cannot shrink from that side — we need a **larger minimum**. So the **only** useful move is to **advance the list that contains the minimum**. When any list runs out of elements we can no longer cover all lists $\rightarrow$ stop.

Heap. Min-heap of `(value, list_id, index)`; the heap gives the *minimum* but we also need the *maximum*, so we maintain `current_max` **separately** (it only ever grows as we push larger values).

current range [min, max]	width	best so far	next move
[0, 5]	5	[0, 5]	pop 0 (list 1)
[4, 9]	5	[0, 5]	pop 4 (list 0)
[5, 10]	5	[0, 5]	pop 5 (list 2)

current range [min, max]	width	best so far	next move
[9, 18]	9	[0, 5]	pop 9 (list 1)
[10, 18]	8	[0, 5]	pop 10 (list 0)
[12, 18]	6	[0, 5]	pop 12 (list 1)
[15, 20]	5	[0, 5]	pop 15 (list 0)
[18, 24]	6	[0, 5]	pop 18 (list 2)
[20, 24]	4	[20, 24] <- new best	pop 20 (list 1)
			list 1 exhausted: stop

```
import heapq

def smallest_range(nums):
    heap = []
    current_max = float('-inf')
    for i, arr in enumerate(nums):
        heapq.heappush(heap, (arr[0], i, 0))
        current_max = max(current_max, arr[0])
    best = [heap[0][0], current_max]
    while True:
        current_min, list_id, index = heapq.heappop(heap)
        if current_max - current_min < best[1] - best[0]:
            best = [current_min, current_max]
        if index + 1 == len(nums[list_id]):
            break
        # one list is exhausted: cannot cover all
        # lists any more
        next_value = nums[list_id][index + 1]
        heapq.heappush(heap, (next_value, list_id, index + 1))
        current_max = max(current_max, next_value)
    return best
```

Complexity. Every element is pushed once → $O(N \log K)$ time, $O(K)$ space.

Mental model. Keep one candidate per sorted source; the heap identifies the current minimum, and advancing that source is the only move that can improve the range.

Also K-way merge in disguise

- **K Smallest Pairs** (§2.10, 1A) — each row `nums1[i] + nums2[*]` is a sorted source.
- **Kth Smallest in a Sorted Matrix** (§2.10, 1B) — each row is a sorted source.

Part 6 · Pattern 5 — Sort + Heap

Recognition. Sort the incoming events by one dimension, then use a heap to manage the currently active set.

This pattern prevents the mistake of thinking "if I see a heap, I should use a heap for everything." Here **sorting does part of the job and the heap does the other part**:

Sorting handles the incoming order; the heap dynamically manages the active set.

6.1 Deriving it: Meeting Rooms II (LeetCode 253)

Problem. Meetings as intervals $[0, 30]$, $[5, 10]$, $[15, 20]$. Find the **minimum number of rooms** so that no two meetings in the same room overlap.

Don't think about heaps yet — if we sort the meetings, what should we sort by, and why?

Common mistake. "Sort by end time, so the meeting ending sooner comes first." — Sorting by end time is useful for *some* interval problems (e.g. maximum number of non-overlapping meetings), but here we care about **when new meetings start**: each time a meeting starts we ask "has any existing room become free before this start?"

So **sort by start time**:

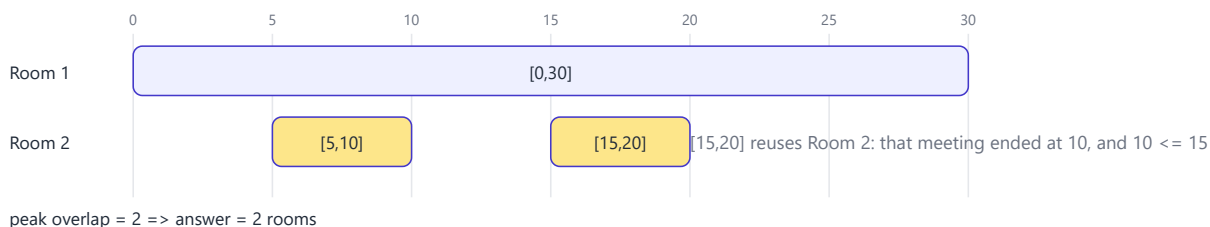
$[0, 30]$ $[5, 10]$ $[15, 20]$

When we reach $[5, 10]$ we ask: *has an existing room become free before time 5?* That is where the heap comes in.

What should the heap contain? Not whole meetings. It should contain **the end times of the rooms that are currently occupied** — the only thing we need to know is *when each occupied room becomes free*. (An early answer, "the current occupied meetings", is close; refine it to *end times of occupied rooms*.)

Why a MIN-heap? We need to find the room that becomes free earliest. If even that one is not free yet, no other room can be free either. Check the earliest-ending room first; the minimum end time is exactly what a min-heap exposes at the root.

Meeting Rooms II: sort by start, heap = end times of occupied rooms



Dry run

Sorted by start: $[0, 30]$, $[5, 10]$, $[15, 20]$

meeting (by start)	check the heap root	heap of end times	rooms in use
[0,30]	no rooms yet: open the first room	[30]	1
[5,10]	earliest end 30 > 5: no free room, open a new one	[10, 30]	2
[15,20]	earliest end 10 <= 15: reuse that room	[20, 30]	2

Answer = 2 rooms.

The invariant and the rule

Invariant. *The heap contains the end times of all currently occupied rooms, and its root is the room that becomes available earliest.*

When a new meeting starts at `start` :

```
if heap and heap[0] <= start:    # the earliest-free room is free by now
    heapq.heappop(heap)         # reuse that room
heapq.heappush(heap, end)       # this meeting occupies a room until 'end'
```

Whether we reused a room or opened a new one, the current meeting occupies **a room until `end`**, so `end` is pushed either way. The answer is the number of rooms ever needed — the **final heap size** here (rooms are never closed once opened; a reused room just replaces its old end time).

A subtle correction. We don't really care *which meeting* ends earliest — we care **which room becomes free earliest**. The distinction becomes important in more complex scheduling problems where rooms have identities or capacities.

Boundary detail. `heap[0] <= start` (not `<`): a meeting that ends at 10 and one that starts at 10 do **not** overlap, so they can share a room.

```
import heapq

def min_meeting_rooms(intervals):
    intervals = sorted(intervals, key=lambda x: x[0])    # by START time
    heap = []                                             # end times of occupied rooms
    for start, end in intervals:
        if heap and heap[0] <= start:                   # earliest-free room is free by now
            heapq.heappop(heap)                         # reuse it
        heapq.heappush(heap, end)                       # this meeting occupies a room until 'end'
    return len(heap)
```

Complexity. Sorting $O(n \log n)$ + at most n push/pop on a heap of size $\leq n \rightarrow O(n \log n)$ time, $O(n)$ space.

Mental model. *Sorting tells me what arrives next; the heap tells me which active resource becomes free first.*

6.2 The same pattern, different words: jobs and machines

Jobs (job, start_time, end_time); each running job needs its own machine; process jobs in start-time order. **What does the heap contain?**

The end times of the jobs that are currently occupying machines. For jobs A [1,10], B [2,7], C [5,12] the heap is [7, 10, 12]. If a new job starts at 8: the minimum end time is 7, $7 \leq 8 \rightarrow$ that machine is free $\rightarrow$ pop 7, assign the machine to the new job, push the new job's end time.

One final insight for Pattern 5.

The sorting determines the order in which things arrive. The heap manages the things that are currently active. Sort the incoming events $\rightarrow$ heap the changing active set.

This pattern appears in Meeting Rooms II, CPU / task scheduling, resource allocation and event simulation.

6.3 Harder problems for Pattern 5

5B. Minimum Number of Servers (peak concurrent jobs)

Jobs [start, end]; a server can run several jobs *one after another* (a server is free once its previous job has finished). How many servers do you need?

Thought process. Same skeleton as meeting rooms — sort by start, heap of end times of active jobs. The difference: for every new job, **remove every server that has become free** (a `while`, not an `if`), push the new end, and the answer is the **maximum heap size ever seen** — the *peak* number of simultaneously active jobs.

job (by start)	machines released	heap of end times	machines busy
A [1,10]	free machines with end ≤ 1 : none	[10]	1
B [2,7]	free machines with end ≤ 2 : none	[7, 10]	2
C [5,12]	free machines with end ≤ 5 : none	[7, 10, 12]	3
D [8,15]	free machines with end ≤ 8 : [7]	[10, 12, 15]	3

Peak = 3 machines (this is `answer = max(answer, len(heap))`).

```
import heapq

def min_servers(jobs):
    jobs = sorted(jobs)
    heap = []
    answer = 0
    for start, end in jobs:
        while heap and heap[0] <= start:
            heapq.heappop(heap)
        heapq.heappush(heap, end)
        answer = max(answer, len(heap))
    return answer
```

Difference from the basic meeting-room problem. Here we explicitly track the **peak** active resource count (`answer = max(answer, len(heap))`) instead of reading the final heap size.

Complexity. $O(n \log n)$ time, $O(n)$ space.

5A. CPU Scheduling — the Single-Threaded CPU (LeetCode 1834)

Tasks have a **start time** (when they become available) and a **duration**. The CPU runs **one task at a time**. When several tasks are available choose the one with the **smallest processing time**, breaking ties by task ID. Return the order in which tasks run. (LeetCode calls this *Single-Threaded CPU*.) Example (start, duration): task 0 (1,2) , task 1 (2,4) , task 2 (3,2) , task 3 (4,1) → order **[0, 2, 3, 1]**.

Thought process. Two *separate* orderings exist:

Question	Answer
When does a task become available?	sort by start time
Which available task runs next?	min-heap by (duration, task_id)

```
sort incoming tasks
    ↓
add every task that has already arrived (start <= time) to the heap
    ↓
min-heap chooses the shortest available task
    ↓
advance time by its duration
    ↓
repeat (if the heap is empty, jump time forward to the next arrival)
```

time	arrived now	waiting heap (after removing the chosen) = (duration, id)	run shortest	finishes at
1	[0]	[]	task 0 (duration 2)	3
3	[1, 2]	[(4, 1)]	task 2 (duration 2)	5
5	[3]	[(4, 1)]	task 3 (duration 1)	6
6	-	[]	task 1 (duration 4)	10

Execution order = [0, 2, 3, 1]

```
import heapq

def schedule_tasks(tasks):
    """tasks = [(start, duration, task_id), ...] (single-threaded CPU)."""
    tasks = sorted(tasks)
    heap = []
    time = 0
    i = 0
    result = []
    while i < len(tasks) or heap:
        if not heap and time < tasks[i][0]:
            time = tasks[i][0]                # CPU idles until the next task arrives
        while i < len(tasks) and tasks[i][0] <= time:
            start, duration, task_id = tasks[i]
            heapq.heappush(heap, (duration, task_id))
            i += 1
        duration, task_id = heapq.heappop(heap)
        time += duration
        result.append(task_id)
    return result
```

Note the `(duration, task_id)` tuple: ties on duration fall through to the task ID, which is exactly the required tie-break.

Recognition. *One ordering determines arrival; another ordering determines which active item gets processed.* — classic **sort + heap**.

Complexity. $O(n \log n)$ time, $O(n)$ space.

Part 7 · Pattern 6 — Heap + Greedy

Recognition. *Candidates become available dynamically; at each step, choose the best currently feasible candidate.*

7.1 Deriving it: what makes this different from Top K?

Suppose you have `tasks = [3, 1, 4, 2]`. Each task has a priority/value, you have a limited number of slots, and at every step you want the **currently highest-value available task**. But after choosing one, **new tasks can become available**. Maximise the total value.

What makes this different from simple Top K?

Let's simplify the problem:

Tasks:

- A: value 10, becomes available at time 1
- B: value 5, becomes available at time 1
- C: value 20, becomes available at time 3
- D: value 8, becomes available at time 3

You can perform **one task at a time**.

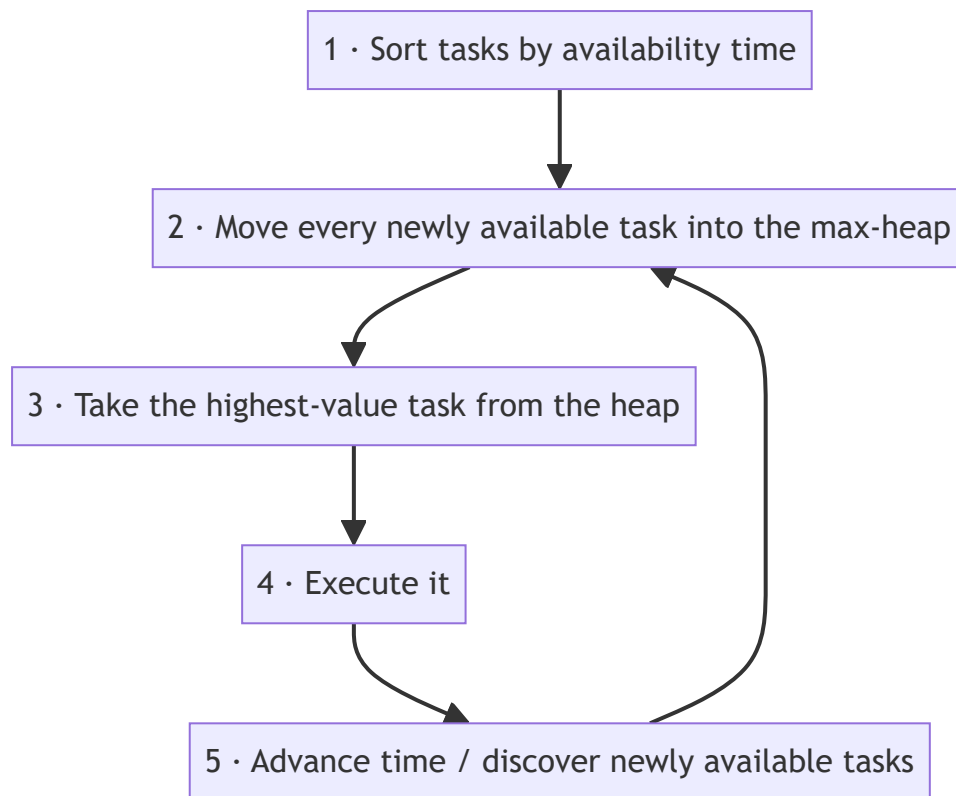
- At time 1 the available tasks are A (10) and B (5) → choose **A**, the highest value.
- Later, at time 3, C and D become available → choose the highest-value task among **everything currently available** (C, plus the leftover B and D).

The important distinction from Top K:

	Top K	Heap + Greedy
Candidates	all available from the beginning	enter the pool dynamically (over time / as your state changes)
Goal	the best K overall	at each step the best currently available one
Your earlier choices	don't affect what is available	do affect what becomes available

If we process tasks in order of their availability time, what should hold the tasks that have become available but haven't been chosen yet? We repeatedly need the **highest-value** available task → a **max-heap**.

The process:



Sorting controls *when* candidates enter the pool; the heap greedily chooses the best candidate from the current pool. **Recognition:** items become available over time and at each step I want the best available item → sort by availability + heap the available candidates + greedily take the best.

7.2 The concrete problem: IPO / Maximum Capital (LeetCode 502)

Problem. You have projects; each has `capital[i]` (the **minimum capital required** to start it) and `profit[i]`. You start with capital `W` and can complete **at most K projects**. Finishing a project **adds its profit** to your capital.

Example: $K = 2$, $W = 0$, project A: capital 0, profit 1; B: capital 1, profit 2; C: capital 1, profit 3. Initially only A is affordable. Take A → $W = 1$. Now B and C are affordable; take the higher profit, C → $W = 4$.

What tracks the currently affordable projects? A max-heap keyed by profit. What determines when a project enters that structure? Its **capital requirement** — a project enters the heap as soon as `capital[i] ≤ W`.

So:

1. **Sort** projects by capital requirement.
2. As `W` increases, add every project with `capital[i] ≤ W` to the max-heap.
3. Take the **maximum-profit** project from the heap.
4. Add its profit to `W`.
5. Repeat up to `K` times.

round	capital W	newly affordable -> pushed	heap of affordable profits (after pop)	action	W after
1	0	(cap 0, profit 1)	[]	take profit 1	1

round	capital W	newly affordable -> pushed	heap of affordable profits (after pop)	action	W after
2	1	(cap 1, profit 2), (cap 1, profit 3)	[2]	take profit 3	4

Final capital = 4

```
import heapq

def find_max_capital(k, w, profits, capital):
    projects = sorted(zip(capital, profits))          # by capital required
    heap = []                                         # max-heap of profits of affordable projects
    i = 0
    for _ in range(k):
        while i < len(projects) and projects[i][0] <= w:
            _, profit = projects[i]
            heapq.heappush(heap, -profit)
            i += 1
        if not heap:
            break
        w += -heapq.heappop(heap)                    # take the most profitable one
    return w
```

The pattern, in one line. Sort by the condition that makes candidates available → put available candidates in a heap → greedily choose the best one.

Complexity. Sorting $O(n \log n)$; every project is pushed at most once and at most K pops → $O((n + K) \log n)$ time, $O(n)$ space.

Why the greedy choice is safe (the proof idea)

Capital is a **threshold, not a cost** — starting a project does not consume capital, it only requires you to *have* it, and finishing it *adds* profit. So among all the projects you can currently afford, taking the **highest-profit** one leaves you with **at least as much capital** as taking any other affordable project. More capital can never make the set of affordable projects *smaller*, so this choice can never hurt any future step.

7.3 Harder problems for Pattern 6

6A. Same IPO, worked with more projects

(The original list files this as "2B. IPO — more general version" under Pattern 2; it is really a Heap + Greedy problem, so it lives here.)

capital = [0,1,1,2], profit = [1,2,3,5], W = 0, K = 3:

current capital → which projects are feasible? → max-heap of feasible profits
→ take the maximum profit → capital increases → more projects become feasible

round	capital W	newly affordable -> pushed	heap of affordable profits (after pop)	action	W after
1	0	(cap 0, profit 1)	[]	take profit 1	1

round	capital W	newly affordable -> pushed	heap of affordable profits (after pop)	action	W after
2	1	(cap 1, profit 2), (cap 1, profit 3)	[2]	take profit 3	4
3	4	(cap 2, profit 5)	[2]	take profit 5	9

Final capital = **9**

The key isn't simply "use a max-heap." It is: *sort to discover what is feasible, heap to choose the best feasible option.*

6B. Maximum Performance of a Team (LeetCode 1383)

Each engineer has `speed[i]` and `efficiency[i]`. Choose **at most k** engineers. Team performance = (sum of speeds) $\times$ (minimum efficiency in the team). Maximise it (return modulo $10^9 + 7$). Example: $n = 6$, `speed = [2,10,3,1,5,8]`, `efficiency = [5,4,3,9,7,2]`, $k = 2 \rightarrow \mathbf{60}$.

The problem looks nothing like Top K at first. The trick is a transformation:

```

sort engineers by efficiency, DESCENDING
    ↓
when processing engineer i, efficiency[i] is the MINIMUM efficiency of the current team
  (everyone already considered has efficiency >= it)
    ↓
so the problem becomes: maximise the SUM OF SPEEDS
    ↓
among engineers seen so far keep the K largest speeds
    ↓
"K largest" → MIN-heap of speeds

```

If the heap exceeds K, remove the **smallest** speed.

engineer (efficiency high -> low)	speeds kept (min-heap)	sum of speeds	min efficiency = current	performance = sum x min-eff
eff 9, speed 1	[1]	1	9	9
eff 7, speed 5	[1, 5]	6	7	42
eff 5, speed 2	[2, 5]	7	5	35
eff 4, speed 10	[5, 10]	15	4	60
eff 3, speed 3	[5, 10]	15	3	45
eff 2, speed 8	[8, 10]	18	2	36

Best = **60**

```

import heapq

def max_performance(n, speed, efficiency, k):
    engineers = sorted(zip(efficiency, speed), reverse=True) # efficiency high -> low
    heap = [] # min-heap of speeds in the team
    speed_sum = 0
    answer = 0
    MOD = 10**9 + 7
    for eff, spd in engineers:
        heapq.heappush(heap, spd)
        speed_sum += spd
        if len(heap) > k:
            speed_sum -= heapq.heappop(heap) # drop the slowest
        answer = max(answer, speed_sum * eff) # eff = minimum efficiency of the team
    return answer % MOD

```

Why this is the real skill. Sort to fix one variable (the minimum efficiency), then use Top-K to optimise the other (the sum of speeds). **Complexity:** $O(n \log n)$.

6C. Minimum Number of Refueling Stops (LeetCode 871)

You start with `startFuel`; the target is at distance `target`; `stations = [(position, fuel), ...]`. One unit of fuel = one unit of distance. What is the **fewest refuelling stops** needed to reach the target (or `-1`)? Example: `target = 100, startFuel = 10, stations = [(10,60), (20,30), (30,30), (60,40)]` → 2.

Thought process. As we travel, stations become *reachable*. We don't have to decide to refuel the moment we pass a station. Instead: **delay the decision**. Collect every station we have driven past; **only when we run out of fuel**, retroactively "refuel" at the station that gave the **most** fuel.

```

stations in order of position
    ↓
add each station we pass to a MAX-heap of fuel amounts
    ↓
when fuel becomes insufficient
    ↓
take the largest fuel available from stations already passed (a stop)

```

station position	what happens	fuel left	max-heap of passed stations' fuel (largest first)
10	drive 10	0	[60], stops=0
20	drive 10; ran short, refuel retroactively with [60]	50	[30], stops=1
30	drive 10	40	[30, 30], stops=1
60	drive 30	10	[40, 30, 30], stops=1
100	drive 40; ran short, refuel retroactively with [40]	10	[30, 30, 0], stops=2

Minimum stops = 2

(The trace's final 0 in the heap is the target treated as a zero-fuel "station" — a trick that lets the same loop handle the final leg.)

```
import heapq

def min_refuel_stops(target, start_fuel, stations):
    stations = list(stations) + [(target, 0)]
    heap = []
    # max-heap (negated) of fuel at stations we
    # passed
    fuel = start_fuel
    prev = 0
    stops = 0
    for position, station_fuel in stations:
        fuel -= position - prev
        while fuel < 0 and heap:
            fuel += -heapq.heappop(heap)
            # retroactively refuel at the best station
            # passed
            stops += 1
        if fuel < 0:
            return -1
        heapq.heappush(heap, -station_fuel)
        prev = position
    return stops
```

Mental model. *Delay the decision. Collect all currently available choices, then when forced to act, take the most valuable one.* One of the most useful heap + greedy ideas.

Complexity. $O(n \log n)$ time, $O(n)$ space.

Part 8 · Pattern 7 — Lazy Deletion (Stale Heap Entries)

Recognition. "The underlying state changes, but removing an arbitrary element from the heap is inconvenient." This is one of the trickier heap patterns.

8.1 Deriving it

Numbers arrive over time and we want the **maximum among the currently active** elements. Initially [10, 7, 15, 4] go into a max-heap. Now 15 becomes **inactive**. The problem: 15 is sitting at the top of the heap, but it is no longer valid.

We could search through the heap and remove it — but **that is not what heaps are good at** (arbitrary removal is $O(n)$, plus you would have to restore the heap property).

If we can't efficiently remove arbitrary elements, what could we do instead? Could we leave the stale element inside the heap and deal with it when it reaches the top?

Yes — that is lazy deletion.

1. Put elements into the heap normally.
2. When an element becomes invalid, **mark it as stale/inactive elsewhere** (a set, a counter, or simply by comparing to the current state).
3. When you look at the heap top, **check whether it is still valid**.
4. If it is **stale** → **discard it** and check the next top.
5. If it is valid → use it.

Heap: [15, 10, 7, 4]

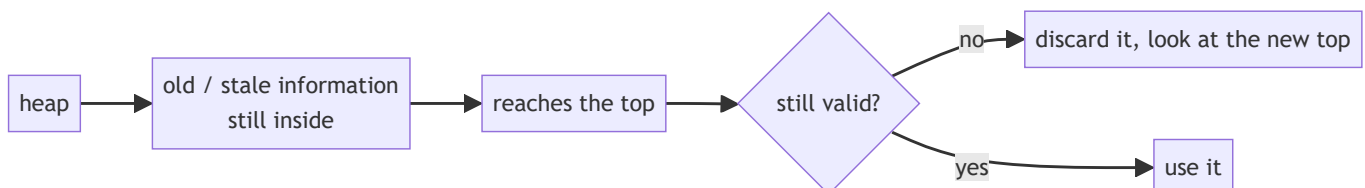
Stale set: {15}

We need the maximum:

top = 15 → stale → remove it

top = 10 → valid → use 10

Why is this safe? Because a stale element **cannot become valid again** in this scenario. So whenever it reaches the top we know it can be discarded permanently.



The key mental model. *Don't pay the cost of removing an arbitrary heap element now. Leave it there and clean it up when it reaches the top.*

Recognition. *"I need a heap, but elements can become invalid / change / be removed while they're somewhere inside it" → lazy deletion: allow stale entries, validate when they reach the top.*

This is especially common with **heaps with priority updates, sliding windows, and repeated changes to candidates**.

8.2 Problem: Sliding Window Maximum (LeetCode 239) — heap version

`nums = [1, 3, -1, -3, 5, 3, 6, 7]` , `k = 3` . For every window of size 3, find the maximum → `[3, 3, 5, 5, 6, 7]` .

One approach is a **max-heap**. But when the window moves, elements **leave** the window while they may still physically sit inside the heap. So each heap entry stores its **index**:

`(value, index)` # stored as `(-value, index)` in Python's min-heap

When the maximum reaches the top: **if its index is outside the current window → discard it**. That is lazy deletion.

i	nums[i]	window	stale tops discarded	top of heap	window max
0	1	[0..0]	-	(v=1, i=0)	-
1	3	[0..1]	-	(v=3, i=1)	-
2	-1	[0..2]	-	(v=3, i=1)	3
3	-3	[1..3]	-	(v=3, i=1)	3
4	5	[2..4]	-	(v=5, i=4)	5
5	3	[3..5]	-	(v=5, i=4)	5
6	6	[4..6]	-	(v=6, i=6)	6
7	7	[5..7]	-	(v=7, i=7)	7

In that run every top happened to be inside the window. A second example where stale entries actually surface at the top (`nums = [9, 1, 2, 3, 4]` , `k = 2`) — watch the "stale tops discarded" column:

i	nums[i]	window	stale tops discarded	top of heap	window max
0	9	[0..0]	-	(v=9, i=0)	-
1	1	[0..1]	-	(v=9, i=0)	9
2	2	[1..2]	(v=9, i=0)	(v=2, i=2)	2
3	3	[2..3]	-	(v=3, i=3)	3
4	4	[3..4]	-	(v=4, i=4)	4

At `i = 2` the heap's top is `(9, i = 0)` , but the window is now `[1..2]` ; index 0 is outside, so it is popped, and only *then* is the real maximum, `2` , exposed.



```

import heapq

def max_sliding_window(nums, k):
    heap = []
    result = []
    for i, x in enumerate(nums):
        heapq.heappush(heap, (-x, i))
        while heap[0][1] <= i - k:                # top has slid out of the window: stale
            heapq.heappop(heap)
        if i >= k - 1:
            result.append(-heap[0][0])
    return result

```

Why stale entries are safe here — the crucial property

Indices only move from valid → invalid. They never become valid again.

The window only slides forward, so an entry whose index is $\leq i - k$ is out **forever**. The moment it reaches the top we discard it and never think about it again. That is *exactly* the condition under which lazy deletion works: **once stale, always stale**.

Notice too that we only inspect the **top**. A stale entry buried deep in the heap does no harm — it simply waits.

Complexity. Every element is pushed once and popped at most once → $O(n \log n)$ time; space $O(n)$ because stale entries linger until they surface.

Aside — the optimal alternative. A monotonic **deque** solves Sliding Window Maximum in $O(n)$. The heap version is worth knowing because the *same lazy-deletion idea* generalises to problems the deque cannot handle (e.g. Sliding Window Median above, or a heap with priority updates).

8.3 Problem: Dijkstra with stale priority-queue entries (LeetCode 743 — Network Delay Time)

This is a particularly important example.

When using a heap for **Dijkstra's shortest paths**, suppose $\text{dist}[A] = 10$, so we push $(10, A)$. Later we discover a **better path**: $\text{dist}[A] = 4$, so we push $(4, A)$. The old $(10, A)$ is **still sitting in the heap**. We don't search for it and remove it. When it eventually reaches the top:

```

heap entry = (10, A)    current dist[A] = 4
10 != dist[A] → it is STALE → discard it

```

Push the new priority and ignore the old one when it becomes stale — instead of updating/removing the old priority. (Python's `heapq` has no "decrease-key", so this is *the* idiom.)

Example graph: $S \rightarrow A$ (10), $S \rightarrow B$ (1), $B \rightarrow A$ (3), $A \rightarrow T$ (1).

pop	check $d == \text{dist}[u]$?	relaxations (pushes)	heap afterwards
(0, S)	valid: relax edges	$\text{dist}[A]=10, \text{dist}[B]=1$	[(1,B), (10,A)]
(1, B)	valid: relax edges	$\text{dist}[A]=4$	[(4,A), (10,A)]
(4, A)	valid: relax edges	$\text{dist}[T]=5$	[(5,T), (10,A)]
(5, T)	valid: relax edges	-	[(10,A)]
(10, A)	STALE ($\text{dist}[A] = 4$): skip	-	[]

Final distances: {'S': 0, 'B': 1, 'A': 4, 'T': 5}

Note the last row: (10, A) reaches the top *after* the better (4, A) was already processed; $10 \neq \text{dist}[A] = 4$, so it is skipped without relaxing any edge.

```
import heapq

def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d != dist[u]:
            # stale entry: a better path was found later
            continue
        for v, weight in graph[u]:
            new_dist = d + weight
            if new_dist < dist[v]:
                dist[v] = new_dist
                heapq.heappush(heap, (new_dist, v))
    return dist
```

Complexity. Each edge can push at most one entry, so the heap holds at most E entries $\rightarrow O((V + E) \log V)$ time ($\log E = O(\log V)$), $O(V + E)$ space.

Why it is the same pattern. The heap can contain *old priority* and *new priority* for the same node. The old entry is stale the instant a better distance is recorded, and — crucially — **once stale, always stale** (distances only decrease).

Recognition summary for Pattern 7.

```
stale entries are allowed
    ↓
don't remove them immediately
    ↓
validate when they reach the top
    ↓
discard if invalid
```

Part 9 · Pattern 8 — Hash Map + Heap (Composite Patterns)

Recognition. *Need frequency/count first, then the top K — or the heap must be combined with a structure that temporarily blocks an item.*

The hash map computes the **score**; the heap maintains the **best K scores** (or the best available choice).

9.1 Top K Frequent Elements — the base case

We solved this in Part 2 (§2.7). It is the simplest instance:

```
Hash map → calculate frequency
+
Min-heap → maintain the  $K$  best frequencies
```

Mental model. *Use the hash map to calculate the score; use the heap to maintain the best K scores. (See §2.7 for the code and the trace.)*

9.2 Harder problems for Pattern 8

8A. Reorganize String (LeetCode 767)

Given `aaabbc`, rearrange the characters so that **no two adjacent characters are equal** (return `""` if impossible).

Thought process. We repeatedly need to choose the **most frequent** remaining character — *but we can't choose the same character we just used*. So a plain max-heap isn't enough:

```
frequency → max-heap
previous character → temporarily unavailable
```

The process for `a:3, b:2, c:1`:

1. Take `a` (most frequent). Now `a` **cannot** be selected again immediately.
2. Take `b` (the best of the rest). Now `a` becomes available again — put it back in the heap.
3. Repeat; the character just used is held back for exactly one step.

take most frequent	then	previous character	result so far
a (had 3)	a: 2 left, held back	-	a
b (had 2)	b: 1 left, held back	a (remaining 2) returns to the heap	ab
a (had 2)	a: 1 left, held back	b (remaining 1) returns to the heap	aba
b (had 1)	b: 0 left, held back	a (remaining 1) returns to the heap	abab

take most frequent	then	previous character	result so far
a (had 1)	a: 0 left, held back	-	ababa
c (had 1)	c: 0 left, held back	-	ababac

Result = **ababac**

```
import heapq
from collections import Counter

def reorganize_string(s):
    freq = Counter(s)
    heap = [(-count, char) for char, count in freq.items()]
    heapq.heapify(heap)
    result = []
    prev_count = 0
    prev_char = ''
    while heap:
        count, char = heapq.heappop(heap)
        result.append(char)
        count += 1
        # one copy used (counts are stored negated)
        if prev_count < 0:
            heapq.heappush(heap, (prev_count, prev_char))
            # previous char may be used again
        prev_count = count
        prev_char = char
    if len(result) != len(s):
        return ""
    return ''.join(result)
```

How the code handles "held back for one step": after popping `char`, we do **not** push it back yet; we remember it as `prev`. On the *next* iteration, before remembering the new character, we push the previous one back (if it still has copies left). So a character is always separated from its previous occurrence by at least one other character.

Feasibility check. A valid arrangement exists exactly when the most frequent character appears at most $\text{ceil}(\text{len}(s) / 2)$ times, i.e. $\text{max_count} \leq (\text{len}(s) + 1) // 2$. If not, the loop ends with fewer characters than $\text{len}(s)$ and the function returns `""`.

Pattern.

```
frequency map
  ↓
max-heap
  ↓
choose best candidate
  ↓
temporarily block the previous candidate
  ↓
choose next best
```

This is **heap + hash map + greedy**. **Complexity:** $O(n \log a)$ with a distinct characters (effectively $O(n)$ for a fixed alphabet).

8B. Task Scheduler (LeetCode 621)

Tasks `A A A B B B`, `cooldown = 2`. The **same task cannot run again until its cooldown expires** (there must be at least `cooldown` other slots — tasks or idles — between two runs of the same task). Find the minimum total time (each task takes 1 unit; the CPU may be idle).

Thought process. At any moment: *among tasks that are currently available, choose the most frequent one*. There are really **two structures**:

```
frequency map
  ↓
max-heap → tasks that are available NOW
  ↓
choose the most frequent available task
  ↓
cooldown queue → tasks that cannot run YET (with the time they become usable)
```

Heap manages what I can choose now; the queue manages what I cannot choose yet.

Each time unit: pop the most frequent available task, run it once (decrement its count), and if it still has copies left put it in the cooldown queue stamped with `time + cooldown`. Before the next unit, if the front of the queue is ready, move it back into the heap. If the heap is empty but the queue is not, the CPU **idles** that unit.

time	runs	available (max-heap) task:remaining	cooling-down queue task:remaining@ready-time	released
1	A	[B:3]	[A:2@t3]	-
2	B	[]	[A:2@t3, B:2@t4]	-
3	idle	[A:2]	[B:2@t4]	A usable again
4	A	[B:2]	[A:1@t6]	B usable again
5	B	[]	[A:1@t6, B:1@t7]	-
6	idle	[A:1]	[B:1@t7]	A usable again
7	A	[B:1]	[]	B usable again
8	B	[]	[]	-

Total time = **8**

```

import heapq
from collections import Counter, deque

def least_interval(tasks, cooldown):
    freq = Counter(tasks)
    heap = [-count for count in freq.values()]          # max-heap of remaining counts (negative)
    heapq.heapify(heap)
    cooldown_queue = deque()                            # (negated remaining count, time it is
usable again)
    time = 0
    while heap or cooldown_queue:
        time += 1
        if heap:
            count = heapq.heappop(heap)                # most frequent available task (negative)
            count += 1                                  # run it once
            if count < 0:
                cooldown_queue.append((count, time + cooldown))
        if cooldown_queue and cooldown_queue[0][1] == time:
            count, _ = cooldown_queue.popleft()
            heapq.heappush(heap, count)
    return time

```

Correction. The code in the original notes stores counts as negatives (for the max-heap) but then does `count = -heapq.heappop(heap)` — which makes `count` *positive* — followed by `count += 1`, which *increases* it, and tests `if count < 0` (never true). The task is then never re-queued and the function returns far too small an answer:

input	correct answer	source's code returns
AAABBB, cooldown 2	8	2
ABC, cooldown 0	3	3

The fix is to keep the popped value **negative**: `count = heapq.heappop(heap); count += 1` (one copy used), and re-push the same negative number. It passed 500/500 random tests against the known closed form, and is what the code above does.

A closed-form check. With `max_count` = the largest task frequency and `n_max` = how many tasks have that frequency:

```
answer = max( len(tasks), (max_count - 1) × (cooldown + 1) + n_max )
```

For AAABBB, cooldown 2: `max(6, (3-1)×3 + 2) = 8` ✓ — a good sanity check for the simulation.

Complexity. Each time unit is $O(\log a)$ with $a \leq 26$ distinct tasks → effectively linear in the total running time.

9.3 Why difficult heap problems combine patterns

Problem	Decomposition
Kth largest	Top K
Sliding window median	Two heaps + lazy deletion
K smallest pairs	K-way merge + Top K
Meeting rooms	Sort + heap
IPO	Sort + heap + greedy
Minimum refuelling stops	Sort / ordered traversal + max-heap + greedy
Dijkstra	Heap + lazy deletion
Task scheduler	Hash map + max-heap + cooldown queue

So don't try to memorise "Problem X → Heap Y." Decompose the problem:

What changes over time?

↓

What choices are currently available?

↓

What choice do I repeatedly need?

↓

What should remain in the heap?

↓

What makes an entry obsolete?

↓

Is another technique needed to determine when candidates enter the heap?

That decomposition is the real skill you're trying to build.

Part 10 · Templates, Decision Guide and Cheat Sheets

10.1 The templates to memorise

These are the templates worth memorising — not individual solutions. Lines marked as pseudo-code (`items` , `is_stale` , ...) are placeholders for problem-specific logic.

Template 1 — Top K largest / smallest

```
import heapq

# --- K largest: MIN-heap of size k -----
heap = []
for x in nums:
    heapq.heappush(heap, x)
    if len(heap) > k:
        heapq.heappop(heap)
# root heap[0] = the k-th largest

# --- K smallest: MAX-heap of size k (negate) -----
heap = []
for x in nums:
    heapq.heappush(heap, -x)
    if len(heap) > k:
        heapq.heappop(heap)
# root -heap[0] = the k-th smallest
```

Template 2 — Repeatedly process the minimum / maximum

```
# minimum
heapq.heapify(items)
while condition:
    x = heapq.heappop(heap)
    # process x
    if new_item:
        heapq.heappush(heap, new_item)

# maximum (negate)
heap = [-x for x in items]
heapq.heapify(heap)
while condition:
    x = -heapq.heappop(heap)
    # process x
    heapq.heappush(heap, -new_item)
```

Template 3 — Two heaps

```
left = []      # max-heap (negated) : smaller half
right = []     # min-heap           : larger half

# insert
if not left or x <= -left[0]:
    heapq.heappush(left, -x)
else:
    heapq.heappush(right, x)

# rebalance
if len(left) > len(right) + 1:
    heapq.heappush(right, -heapq.heappop(left))
elif len(right) > len(left):
    heapq.heappush(left, -heapq.heappop(right))

# median: -left[0] if len(left) > len(right) else (-left[0] + right[0]) / 2
```

Think: smaller half | larger half = max-heap | min-heap.

Template 4 — K-way merge

```
heap = []
for i, arr in enumerate(arrays):
    if arr:
        heapq.heappush(heap, (arr[0], i, 0))

while heap:
    value, source, index = heapq.heappop(heap)
    # use value
    next_index = index + 1
    if next_index < len(arrays[source]):
        heapq.heappush(heap, (arrays[source][next_index], source, next_index))
```

Think: one candidate per source → heap → global minimum → advance that source.

Template 5 — Sort + Heap

```
items.sort(key=lambda x: x.start)
heap = []

for item in items:
    # remove / reuse resources that are no longer active
    while heap and heap[0] <= item.start:
        heapq.heappop(heap)
    # current item becomes active
    heapq.heappush(heap, item.end)
```

The exact cleanup condition depends on the problem (use `if` to *reuse* one resource as in Meeting Rooms II, or `while` to *release all* freed resources as in the peak-servers variant).

Template 6 — Heap + Greedy

```
items.sort(key=lambda x: x.availability)
heap = []
i = 0

while decision_limit_not_reached:
    while i < n and items[i].is_available():
        heapq.heappush(heap, items[i].score)      # (negate for max-heap)
        i += 1
    if not heap:
        break
    best = heapq.heappop(heap)
    # greedily choose best
```

Think: `sort` → discover feasible choices; `heap` → choose the best feasible choice.

Template 7 — Lazy deletion

```
heap = []
for i, value in enumerate(items):
    heapq.heappush(heap, (-value, i))
    while heap and is_stale(heap[0]):
        heapq.heappop(heap)
    current_best = -heap[0][0]
```

The important part isn't the exact code — it is:

stale entries are allowed → don't remove them immediately → validate when they reach the top → discard if invalid

Template 8 — Hash map + Heap

```
freq = Counter(items)          # hash map: the score
heap = []
for key, count in freq.items():
    heapq.heappush(heap, (count, key))      # min-heap keeps the K best
    if len(heap) > k:
        heapq.heappop(heap)
```

10.2 Final heap decision guide



If you need...	Use
repeated min / max	HEAP
only the K best elements	HEAP OF SIZE K
the median dynamically	TWO HEAPS
multiple sorted sources	K-WAY MERGE
events in an order, active items changing	SORT + HEAP
choices feasible over time, best feasible one	HEAP + GREEDY
elements invalid but arbitrary deletion is inconvenient	LAZY DELETION
frequency/count first, then top K	HASH MAP + HEAP

The biggest thing to retain is not the Python syntax. The goal is to recognise the *underlying operation* rather than think "I see a heap, therefore use a heap." The question to ask on a new problem:

"What information do I repeatedly need from a changing set, and can a heap make exactly that operation cheap?"

10.3 Which heap? — direction cheat sheet

Situation	Heap	Store in Python
K largest	min-heap of size K	<code>x</code>
K smallest	max-heap of size K	<code>-x</code>
K most frequent	min-heap of size K on frequency	<code>(count, item)</code>
K closest	max-heap of size K on distance	<code>(-dist, ...)</code>
Lower half (median)	max-heap	<code>-x</code>
Upper half (median)	min-heap	<code>x</code>
Two smallest ropes	min-heap	<code>x</code>
Two heaviest stones	max-heap	<code>-x</code>
K-way merge	min-heap	<code>(value, source, index)</code>
Occupied rooms / servers	min-heap of end times	<code>end</code>
Affordable projects (best profit)	max-heap	<code>-profit</code>
Ladders on largest climbs	min-heap (size = ladders)	<code>climb</code>
Refuelling: the station with the most fuel	max-heap of fuel	<code>-fuel</code>

Situation	Heap	Store in Python
Dijkstra	min-heap on distance	<code>(dist, node)</code>
Most frequent available char/task	max-heap on count	<code>(-count, item)</code>

10.4 Complexity cheat sheet

Problem	Time	Space
Kth largest / smallest	$O(n \log k)$	$O(k)$
Top K frequent	$O(n + d \log k)$	$O(d + k)$
K closest points	$O(n \log k)$	$O(k)$
K smallest pairs	$O(k \log k)$	$O(k)$
Kth smallest in sorted matrix	$O(k \log n)$	$O(n)$
Median from a stream	add $O(\log n)$, median $O(1)$	$O(n)$
Sliding window median	$O(n \log n)$	$O(n)$
Connect ropes	$O(n \log n)$	$O(n)$
Last stone weight	$O(n \log n)$	$O(n)$
Min cost to hire K workers	$O(n \log n)$	$O(n)$
Furthest building	$O(n \log L)$	$O(L)$
Merge K sorted lists / arrays	$O(N \log K)$	$O(K)$
Smallest range from K lists	$O(N \log K)$	$O(K)$
Meeting rooms II / min servers	$O(n \log n)$	$O(n)$
Single-threaded CPU	$O(n \log n)$	$O(n)$
IPO	$O((n + K) \log n)$	$O(n)$
Max performance of a team	$O(n \log n)$	$O(n)$
Min refuelling stops	$O(n \log n)$	$O(n)$
Sliding window maximum (heap)	$O(n \log n)$	$O(n)$
Dijkstra	$O((V + E) \log V)$	$O(V + E)$
Reorganize string	$O(n \log a)$	$O(a)$
Task scheduler	$\approx O(\text{total time} \cdot \log a)$	$O(a)$

10.5 Python pitfalls checklist

Pitfall	Fix
<code>heapq</code> is a min -heap only	negate for a max-heap (Python 3.14 adds <code>*_max</code> functions)
Forgetting to negate back when reading	<code>-heap[0]</code> , <code>-heappop(heap)</code>

Pitfall	Fix
Ties compare the next tuple element; objects aren't comparable	add a unique tie-breaker <code>(key, i, obj)</code>
<code>heapify</code> mutates the list you pass in	copy first if the caller still needs it
<code>len(heap)</code> counts stale entries in lazy-deletion code	keep separate <i>valid</i> size counters (Sliding Window Median)
Reading <code>heap[0]</code> on an empty heap → <code>IndexError</code>	guard with <code>if heap</code>
Off-by-one when the heap should hold exactly K	push first, then <code>if len(heap) > k: pop</code>
<code><</code> vs <code><=</code> in interval problems	<code>end <= start</code> means back-to-back meetings can share a room
Sorting by the wrong key (e.g. end time for Meeting Rooms II)	sort by the key that says <i>when things arrive</i>
Storing floats (<code>w / q</code>) as heap keys	ties/precision — prefer integer keys where possible

10.6 Which patterns combine?

Combined problem	Patterns
K smallest pairs, Kth smallest in matrix	K-way merge + Top K
Sliding window median	Two heaps + lazy deletion
Sliding window maximum, Dijkstra	Heap + lazy deletion
Min cost to hire K workers	Sort + Top K (max-heap) + greedy
Max performance of a team	Sort + Top K (min-heap) + greedy
IPO, Min refuelling stops	Sort + heap + greedy
Reorganize string, Task scheduler	Hash map + heap (+ cooldown queue)
Furthest building	Greedy + heap (replace the cheapest earlier choice)

Part 11 · Practice, Recognition Drills and Interview Q&A

11.1 Practice list (LeetCode)

From this guide

Pattern	Problems
1 Top K	215 Kth Largest Element · 973 K Closest Points to Origin · 347 Top K Frequent Elements · 373 Find K Pairs with Smallest Sums · 378 Kth Smallest Element in a Sorted Matrix
2 Two heaps	295 Find Median from Data Stream · 480 Sliding Window Median
3 Repeatedly process	1167 Minimum Cost to Connect Sticks (Connect Ropes) · 1046 Last Stone Weight · 857 Minimum Cost to Hire K Workers · 1642 Furthest Building You Can Reach
4 K-way merge	23 Merge k Sorted Lists · 632 Smallest Range Covering Elements from K Lists
5 Sort + heap	253 Meeting Rooms II · 1834 Single-Threaded CPU
6 Heap + greedy	502 IPO · 1383 Maximum Performance of a Team · 871 Minimum Number of Refueling Stops
7 Lazy deletion	239 Sliding Window Maximum · 743 Network Delay Time (Dijkstra)
8 Hash map + heap	347 Top K Frequent Elements · 767 Reorganize String · 621 Task Scheduler

Further practice (not in the original session — suggestions)

703 Kth Largest Element in a Stream · 692 Top K Frequent Words · 1353 Maximum Number of Events That Can Be Attended (sort + heap) · 218 The Skyline Problem (sort + heap + lazy deletion) · 358 Rearrange String k Distance Apart (hash map + heap + queue) · 355 Design Twitter (K-way merge) · 407 Trapping Rain Water II (heap-driven frontier).

11.2 Recognition drills

Cover the right-hand column and name the pattern and the heap for each scenario.

Scenario	Pattern → heap
Find the 2nd largest of [3, 2, 1, 5, 6, 4] for a very large n	Top K → min-heap of size 2
The 3rd smallest of [7, 10, 4, 3, 20, 15]	Top K → max-heap of size 3
The 2 most frequent elements	Hash map + Top K → min-heap on frequency
The 2 points closest to the origin	Top K → max-heap on distance ²
Median after every insertion of millions of numbers	Two heaps → max-heap (lower) + min-heap (upper)
Connect all ropes at minimum total cost	Repeatedly process min → min-heap
Two heaviest stones collide repeatedly	Repeatedly process max → max-heap
A processor always runs the shortest remaining task	Repeatedly process min → min-heap
Take the smallest number, double it, put it back, ×3	Repeatedly process min → min-heap, push modified value back

Scenario	Pattern → heap
100 sorted files → one global sorted stream	K-way merge → min-heap of one candidate per file
Minimum meeting rooms	Sort by start + min-heap of end times
Jobs need machines while running	Sort + heap of end times of active jobs
Projects unlock as capital grows; maximise final capital	Sort by capital + max-heap of profit (greedy)
Max over each sliding window, elements slide out	Max-heap + lazy deletion (or a monotonic deque)
Shortest paths with an improving distance table	Min-heap + lazy deletion (Dijkstra)
No two equal neighbours in the rearranged string	Hash map + max-heap , hold back the previous char
Same task can't repeat until a cooldown expires	Hash map + max-heap + cooldown queue

11.3 Interview questions and short answers

Q. What is a heap and what does it guarantee? A complete binary tree stored in an array where every parent is $\leq$ (min-heap) or $\geq$ (max-heap) its children. It guarantees only that the **root is the minimum/maximum** — not that the rest is sorted.

Q. Complexities of the heap operations? Peek $O(1)$; insert $O(\log n)$ (sift up); extract-min/max $O(\log n)$ (sift down); heapify a whole array $O(n)$.

Q. Why is building a heap $O(n)$ rather than $O(n \log n)$? Bottom-up heapify sifts each parent down; half the nodes are leaves (0 work), a quarter can move 1 level, an eighth 2 levels... the sum $n \cdot \sum k/2^{k+1}$ converges to $O(n)$.

Q. K largest elements — which heap and why? A **min-heap of size K**: the root is the weakest of the K largest, so it is the one to evict when a bigger element arrives. Time $O(n \log k)$, space $O(k)$.

Q. Heap or sort for Top K? Sorting is $O(n \log n)$; a size-K heap is $O(n \log k)$ and works on streams with $O(k)$ memory. Quickselect is $O(n)$ average for a static array.

Q. How do you maintain a running median? Two heaps — a max-heap for the smaller half and a min-heap for the larger half — sizes differing by at most 1. The median is the root of the bigger heap, or the average of both roots. Insert $O(\log n)$, median $O(1)$.

Q. Why is the smallest-first strategy right for connecting ropes? Each merge's result is counted again in later merges; smaller ropes should be counted more often and larger ropes fewer times. It is Huffman's algorithm.

Q. What is an invariant, and why does it matter for heap problems? A condition kept true throughout (e.g. "*the max-heap holds exactly the remaining stones*"). It tells you what is inside the heap and therefore when to push, pop or ignore.

Q. How does K-way merge work and what does it cost? Push the first element of each sorted source as (value, source, index); repeatedly pop the minimum, output it and push that source's next element. $O(N \log K)$ time, $O(K)$ space.

Q. Why sort by start time, not end time, for Meeting Rooms II? Because we decide at each meeting's *start* whether an existing room is free. The heap of end times gives the earliest-free room; if even that isn't free by *start*, no room is.

Q. What is lazy deletion and when is it safe? Leave invalid entries in the heap and discard them only when they reach the top, after checking validity. Safe when an entry, once invalid, **never becomes valid again** (sliding-window indices, outdated Dijkstra distances).

Q. Why does Dijkstra with a heap push duplicates? `heapq` has no decrease-key. We push the improved `(dist, node)` and skip any popped entry whose distance no longer equals `dist[node]`.

Q. When *not* to use a heap? When you only need the sorted order once (just sort), when you need arbitrary deletion or search (use a balanced tree / hash map), or when a simpler structure such as a monotonic deque or a sliding window solves it in linear time.

Q. How do you do a max-heap in Python? Store negated keys (`heappush(h, -x)` ; read `-h[0]`), or on 3.14+ use the `*_max` functions. For tuples negate the sort key.

11.4 One-page recap

- **Heap** = fast access to one extreme of a changing set. Root only is guaranteed; $O(1)$ peek, $O(\log n)$ push/pop, $O(n)$ heapify.
- **Top K** — size-K heap; direction = *who do I evict?* (K largest $\rightarrow$ min-heap; K smallest $\rightarrow$ max-heap). $O(n \log k)$.
- **Two heaps** — max-heap (lower half) + min-heap (upper half), sizes within 1; roots straddle the median.
- **Repeatedly process** — pop the extreme, modify, push the result back; the invariant is "the heap holds what remains".
- **K-way merge** — one candidate per sorted source; `(value, source, index)`; $O(N \log K)$.
- **Sort + heap** — sort by arrival; heap of end times of the active set; reuse when `end <= start`.
- **Heap + greedy** — sort by feasibility; heap of feasible candidates; take the best; delay decisions.
- **Lazy deletion** — allow stale entries; validate at the top; safe if stale stays stale.
- **Hash map + heap** — count first; heap the counts; a queue can hold blocked items.
- **Decompose, don't memorise:** *What changes over time? What choices are available? What do I repeatedly need? What stays in the heap? What makes an entry obsolete? What decides when candidates enter?*